



Tribhuvan University
INSTITUTE OF SCIENCE AND TECHNOLOGY (IOST)

A PROJECT REPORT
ON
“AUTOMATED MULTI-BRANCH ENTERPRISE NETWORK”

SUBMITTED BY:
MR. AAYUSH CHAPAGAIN (SYMBOL NO. 72012062)
MR. SANCHHEP RAJ SHRESTHA (SYMBOL NO. 72012077)

*In Partial Fulfillment of the Requirements for Bachelors of Science in Computer Science
and Information Technology (Bsc.CSIT)*

SUBMITTED TO
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY
LUMBINI CITY COLLEGE
TILOTTAMA-04, RUPANDEHI, NEPAL

UNDER THE SUPERVISION OF
MR. KAMAL BAHADUR THAPA

MAY, 2026

STUDENT’S DECLARATION

We hereby declare that the project report entitled “**Automated Multi Branch Enterprise Network**” submitted in partial fulfillment of the requirement for the Bachelor’s degree of Bachelor in Computer Science and information Technology from Tribhuvan University is our original work and not submitted for the award of any other degree, diploma, fellowship, or any other similar title or prize.

Aayush Chapagain (SYMBOL NO.79012062)

Sanchhep Raj Shrestha (SYMBOL NO.79012077)

ACKNOWLEDGMENT

First of all, we would like to express our sincere thanks to the Department of **CSIT** for providing us a golden opportunity to carry out this project work on the “**Automated Multi Branch Enterprise Network**”. We extend our sincere and heartfelt thanks to our project supervisor, **MR. KAMAL BAHADUR SHRISH**, for his continuous guidance, constructive feedback, unwavering support, and expert suggestions throughout the course of this project. His encouragement and technical insight were indispensable in shaping this work to its current form.

Our heartfelt gratitude goes to **Mr. Sujan Dhakal** coordinator for his constructive feedback and support. We also express a deep sense of gratitude to our teachers, **Mr. Rahul Shakya and Mr. Bhuban Panthee** for their suggestions and guidance inspiration cooperation in an academic perspective.

Finally, we express our deepest appreciation to our families and friends for their constant motivation and support throughout this academic journey.

ABSTRACT

This project presents the design, implementation, and validation of an “Automated Multi-Branch Enterprise Network” a production-grade network infrastructure built to address the operational challenges of managing geographically distributed branch offices. The system was implemented in PNetLab using Cisco IOS virtual devices in a three-tier hierarchical architecture spanning one Headquarters and two Branch offices with 26 network devices. OSPF Area 0 provides dynamic routing with automatic convergence, while IPsec Site-to-Site VPN with AES-256 encryption delivers secure WAN failover activating within 15 seconds of primary link failure. HSRP version 2 ensures gateway redundancy, EtherChannel with LACP provides link aggregation, and VLANs 10/20/30/99 segment traffic across all sites. Three custom Python algorithms were developed and applied to the actual network topology. Dijkstra's shortest path algorithm, Bellman-Ford algorithm for result validation, and a Weighted Health Scoring Algorithm computing composite device health scores every 60 seconds. All results are stored in MySQL with automated GitHub cloud backup. A Flask-based NOC dashboard provides six functional modules including live health monitoring, browser-based SSH execution, interactive path analysis, and real-time syslog streaming. All 13 test cases passed successfully with 100% cross-site ping success, validated VPN failover, and identical optimal paths from both algorithms confirming implementation correctness. The project demonstrates that a fully automated, secure, and observable enterprise network can be built using open-source tools and industry-standard protocols.

Keywords: *Enterprise Networking, OSPF, IPsec VPN, HSRP, Dijkstra's Algorithm, Bellman Ford Algorithm, NOC Dashboard, Network Automation, PNetLab, Python, Flask, MySQL*

TABLE OF CONTENTS

STUDENT'S DECLARATION	ii
ACKNOWLEDGMENT	iii
ABSTRACT	iv
LIST OF ABBREVIATIONS	viii
LIST OF FIGURES	x
LIST OF TABLES	xi
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.5 Development Methodology.....	3
1.6 Report Organization	4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	6
2.1 Background Study	6
2.2 Literature Review	6
2.2.1 Traditional Enterprise Network Architectures	7
2.2.2 Contemporary Enterprise Network Solutions	7
2.2.3 Network Automation.....	8
2.2.4 Graph-Based Routing Algorithms	8
2.2.5 Network Monitoring.....	8
CHAPTER 3: SYSTEM ANALYSIS	9
3.1 Research Methodology	9
3.1.1 Emulation Environment (PNETLab)	9
3.1.2 Automation with Python	9
3.2 Network Analysis	10
3.2.1 Requirement Analysis	10
3.2.2 Functional Requirements	10
3.2.3 Non-Functional Requirements	11
3.3 Feasibility Study.....	11

3.3.1 Technical Feasibility	11
3.3.2 Operational Feasibility	12
3.3.3 Economic Feasibility	12
3.3.4 Schedule Feasibility	12
3.4 System Architecture	13
CHAPTER 4: NETWORK DESIGN AND IMPLEMENTATION	16
4.1 Introduction.....	16
4.2 IP Addressing Scheme	17
4.3 OSPF Routing Configurations	20
4.4 IPsec VPN Configuration.....	21
4.5 HSRP and EtherChannel	22
4.6 Description of Algorithms.....	23
4.6.1 Dijkstra's Shortest Path Algorithm	23
4.6.2 Bellman-Ford Algorithm	23
4.6.3 Weighted Health Scoring Algorithm	24
4.7 MySQL Database	25
4.8 NOC Dashboard	26
4.8.1 Overview Module	26
4.8.2 Command Center Module	26
4.8.3 Path Analysis Module	26
4.8.4 Configuration Backup Module.....	27
4.8.5 Real-time Syslog Viewer	27
CHAPTER 5: IMPLEMENTATION AND TESTING	28
5.1 Implementation	28
5.1.1. Network Design and Emulation Tools	28
5.1.2 Networking Protocols and Technologies.....	28
5.1.3. Automation Tools	30
5.1.4. Monitoring and Logging Tools	30
5.1.5. Database and Backend Tools.....	31
5.1.6. Testing and Validation Tools	31
5.1.7. Documentation Tools	33

5.2 Testing Methodology	33
5.3 Test Cases and Results	34
5.3.1 Connectivity Testing	34
5.3.2 VPN Failover Testing.....	34
5.3.3 Health Scoring Results	35
5.4 Result Analysis.....	35
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS	36
6.1 Conclusion	36
6.2 Future Recommendations	37
REFERENCES.....	38
APPENDICES	39

LIST OF ABBREVIATIONS

ACLs	Access Control Lists
AES	Advanced Encryption Standard
APIs	Application Programming Interfaces
ASW	Access-Layer Switch
BPDU	Bridge Protocol Data Unit
CCNP	Cisco Certified Network Professional
CEF	Cisco Express Forwarding
CIDR	Classless Inter-Domain Routing
CLI	Command Line Interface
CPU	Central Processing Unit
CR	Core Router
CSV	Comma Separated Values
DHCP	Dynamic Host Configuration Protocol
DSW	Distribution-Layer Switch
HQ	Headquarters
HSRP	Hot Standby Router Protocol
IDC	Internet Data Center
ICMP	Internet Control Message Protocol
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
IPsec	Internet Protocol Security
IPv4	Internet Protocol Version 4
IPv6	Internet Protocol Version 6
IOS	Cisco Internetwork Operating System
ISP	Internet Service Provider
IOS-XE	Internetwork Operating System Extended Edition
IT	Information Technology
LACP	Link Aggregation Control Protocol
LSDB	Link-State Database
LSR	Label Switching Router

MAC	Media Access Control
MySQL	My Structured Query Language
NAT	Network Address Translation
NOC	Network Operations Center
OSPF	Open Shortest Path First
PAT	Port Address Translation
PCs	Personal Computers
PNetLab	Packet Network Emulator Tool Lab
POST	Power On Shelf Test
PPDIOO	Plan, Prepare, Design, Implement, Operate, Optimize
QM Idle	Quick Mode Idle
REST APIs	Representational State Transfer Application Programming Interfaces
RFC	Request for Comments
RIP	Routing Information Protocol
RSTP	Rapid Spanning Tree Protocol
RTT	Round-Trip Time
SA Lifetime	Security Association Lifetime
SCP	Secure Copy Protocol
SDN	Software Defined Networking
SD WAN	Software Defined Wide Area Network
SSH	Secure Shell
SNMP	Simple Network Management Protocol
SPF	Shortest Path First
STP	Spanning Tree Protocol
TCP	Transmission Control Protocol
TFTP	Trivial File Transfer Protocol
UDP	User Datagram Protocol
VLAN	Virtual Local Area Network
VM	Virtual Machine
VPN	Virtual Private Network
VRRP	Virtual Router Redundancy Protocol
WAN	Wide Area Network
ZTP	Zero Touch Provisioning

LIST OF FIGURES

Figure 1.1: Cisco Network Lifecycle Management	4
Figure 3.1: Timeline	12
Figure 3.2: Gantt Chart	13
Figure 3.3: System Architecture	14
Figure 3.4: Logical Network Topology Diagram	15
Figure 4.1: OSPF Routing Configuration on HQ-CR1	20
Figure 4.2: OSPF Neighbor Adjacency Verification	20
Figure 4.3: IPsec VPN Configuration on HQ-CR1	21
Figure 4.4: IPsec VPN Tunnel Status Verification	21
Figure 4.5: HSRP Status Verification on HQ-DSW1	22
Figure 4.6: EtherChannel LACP Status Verification on HQ-DSW1	22
Figure 4.7: Enterprise Network Health Score Monitoring Output	25

LIST OF TABLES

Table 3.1: Gantt Chart Schedule	12
Table 4.1: Network Address Allocation Overview	17
Table 4.2: VLAN and Subnet Allocation Overview	17
Table 4.3: Headquarters Network Device IP addressing Overview	17
Table 4.4: Branch A Network Device IP Addressing Overview	18
Table 4.5: Branch B Network Device IP Addressing Overview	19
Table 4.6: ISP Cloud Layer IP Addressing Overview	19
Table 4.7: WAN Link and IPsec VPN Connectivity Overview	20
Table 4.8: IPsec VPN Security Parameters and Peers Configuration	21
Table 4.9: MySQL Database Schema Overview	25
Table 5.1: Connectivity Testing Results	34
Table 5.2: VPN Failover Testing Results	34

CHAPTER 1: INTRODUCTION

1.1 Introduction

In the modern digital era, enterprise organizations rely heavily on robust, secure, and scalable network infrastructures to support business operations, remote connectivity, and data-driven decision-making. As organizations expand geographically, multi-branch network architectures have become a necessity rather than a choice. These distributed environments introduce significant challenges related to availability, security, management complexity, and operational efficiency.

Traditional network deployments that depend on manual configuration and limited monitoring are increasingly insufficient to meet current operational demands. Network automation, centralized monitoring, and data-driven management have emerged as critical components of next-generation enterprise networks. Technologies such as dynamic routing protocols, secure WAN connectivity, high-availability mechanisms, and automation using scripting languages are now widely adopted across industries worldwide.

This project presents the complete manual design, configuration, implementation, and validation of an Automated Multi-Branch Enterprise Network. Every network device was manually configured through Cisco IOS CLI including routing protocols, VPN parameters, VLAN segmentation, NAT/PAT, ACLs, and redundancy mechanisms reflecting real-world enterprise network engineering practices. The system further integrates Python-based automation, graph-based path computation, real-time monitoring, and a custom NOC dashboard. The entire infrastructure was implemented and validated in PNetLab using Cisco IOS virtual devices, aligned with Cisco standards and international industry best practices.

1.2 Problem Statement

Organizations with multiple geographically distributed branch offices consistently face operational challenges that directly impact network reliability, security, and administrative efficiency. Without a properly designed and standardized network architecture, managing connectivity across multiple sites becomes increasingly complex and error-prone.

Secure communication between branches and headquarters over public networks is a critical concern, as data transmitted without proper encryption mechanisms is

vulnerable to interception and unauthorized access. Organizations that rely on basic internet connectivity without VPN or dedicated leased lines expose sensitive business data to significant security risks.

Network administrators frequently lack real-time visibility into the operational status of remote branch devices, making proactive fault detection extremely difficult. The absence of centralized monitoring forces administrators into reactive troubleshooting workflows dependent on user-reported problems, increasing mean time to resolution and reducing overall network reliability.

Without standardized network configurations and automated monitoring tools, maintaining consistency across distributed sites becomes a major challenge. Configuration drift, inconsistent security policies, and absence of real-time health metrics collectively degrade network performance and increase operational complexity. Furthermore, the absence of automated path computation and network intelligence means that routing decisions and failover mechanisms cannot be mathematically validated or optimized, leaving the network vulnerable to suboptimal traffic paths and undetected failures.

This project addresses each of these challenges through a systematic manual CLI-based configuration of all 26 network devices using industry-standard protocols, combined with Python-based automation, graph-based shortest path algorithms, and a real-time centralized NOC dashboard for comprehensive network visibility and control.

1.3 Objectives

The primary objective of this project is to design and implement an automated, secure, and scalable multi-branch enterprise network that aligns with contemporary enterprise networking standards. The specific objectives of the project are as follows:

- Design a scalable and highly available multi-branch network.
- Implement secure WAN connectivity with centralized monitoring.
- Automate network configuration and management.

1.4 Scope and Limitation

The scope of this project encompasses the complete manual design, CLI-based configuration, implementation, and validation of an Automated Multi-Branch Enterprise Network within a PnetLab virtual emulation environment. The project implements a three-tier hierarchical architecture consisting of one Headquarters and

two Branch offices interconnected via primary leased lines and IPsec Site-to-Site VPN failover, with a total of 26 Cisco IOS network devices manually configured through direct console access.

The implementation covers a complete enterprise protocol stack including OSPF Area 0 for dynamic routing, IPsec VPN with AES-256 encryption for secure WAN failover, HSRP version 2 for gateway redundancy, EtherChannel LACP for link aggregation, VLAN segmentation (10/20/30/99), 802.1Q trunking, NAT/PAT, extended ACLs, DHCP, SSH version 2, and STP Rapid-PVST. Python-based automation handles real-time health monitoring, shortest path computation using Dijkstra's and Bellman-Ford algorithms, and automated database-backed cloud backup. A Flask-based NOC dashboard with six functional modules provides centralized network visibility, browser-based command execution, and real-time syslog monitoring. A MySQL database stores device inventory, health scores, alerts, and monitoring logs, with automated GitHub synchronization every six hours. The entire system was rigorously tested across 13 structured test cases validating connectivity, security, redundancy, algorithm correctness, and monitoring functionality.

Despite its advantages, the system has certain limitations which are given as follows:

- **Virtual Environment Dependency:** The network operates within PNetLab emulation; physical hardware deployment would require vendor-specific hardware compatibility verification and additional configuration adjustments.
- **Local Dashboard Access:** The NOC dashboard is accessible only within the local lab network at <http://10.1.10.50:5000> external access would require HTTPS implementation and token-based authentication.
- **Technical Complexity:** Manual CLI configuration of 26 Cisco IOS devices requires strong networking knowledge and careful attention to protocol dependencies, particularly NAT exclusion ordering for IPsec VPN.

1.5 Development Methodology

Cisco Network Lifecycle Management is structured around the PPDIOO framework (Prepare, Plan, Design, Implement, Operate, Optimize), designed to improve network availability, reduce expenses, and boost business agility.

- **Prepare:** Identified requirements, assessed constraints, established project scope.

- **Plan:** Developed IP addressing scheme, protocol selection, and Gantt chart timeline.
- **Design:** Designed three-tier architecture, VLAN scheme, VPN design, and automation framework.
- **Implement:** Configured all 26 devices, deployed Python scripts, built NOC dashboard.
- **Operate:** Validated VPN failover, HSRP redundancy, health scoring, and syslog monitoring.
- **Optimize:** Refined NAT exclusion rules, resolved CEF forwarding bug, tuned auto-refresh intervals.



Figure 1.1: Cisco Network Lifecycle Management

1.6 Report Organization

This report is divided into six chapters. Each chapter is further divided into different sections to provide a clear understanding of the project. The preliminary section contains overall information about the project, including abstract, table of contents, list of figures, and abbreviations.

Chapter 1: This chapter introduces the Automated Multi-Branch Enterprise Network project, including its objectives, scope, problem statement, and significance.

Chapter 2: This chapter presents the theoretical background, related technologies, and review of previous studies relevant to enterprise networking and automation.

Chapter 3: This chapter explains the system requirements, feasibility analysis, network architecture, and overall design of the proposed system.

Chapter 4: This chapter describes the implementation process, including hardware and software tools, network configuration, and deployment procedures.

Chapter 5: This chapter discusses the testing methods used to verify system performance, security, reliability, and overall functionality.

Chapter 6: This chapter summarizes the project outcomes and provides recommendations for future improvements and enhancements.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Enterprise networking has evolved significantly from simple local area networks to complex, geographically distributed infrastructures. In the past, organizations relied on static, manually configured networks that were difficult to manage and scale. As businesses grew to include multiple branch offices, the traditional hub-and-spoke model became a standard, but it introduced inefficiencies like suboptimal routing and single points of failure.

The rise of the internet and the need for secure inter-branch communication led to the adoption of Virtual Private Networks (VPNs). However, managing these VPNs, along with routing protocols, access control lists, and device configurations, across dozens of locations became a significant operational burden for IT teams. This burden often resulted in configuration errors, security inconsistencies, and prolonged downtime.

In recent years, the focus has shifted towards automation and centralized management. The advent of software-defined networking (SDN) and SD-WAN has offered promising solutions for dynamic traffic management and simplified operations, but these technologies often come with high costs and vendor lock-in [1]. Concurrently, the open-source community has developed powerful tools like Python, Ansible, and Netmiko, enabling network engineers to automate repetitive tasks, enforce configuration standards, and build custom monitoring solutions. This project leverages these modern tools and concepts to build an enterprise-grade network solution that balances cost, security, and operational efficiency within a robust, emulated environment.

2.2 Literature Review

A comprehensive literature review is conducted on enterprise network design models, including hierarchical network architecture, secure WAN technologies, and network automation frameworks. Existing systems often rely on manual configuration and decentralized monitoring, leading to inefficiencies and operational risks. Industry standards such as Cisco Enterprise Architecture, OSPF routing, IPsec VPNs, SNMP-

based monitoring, and database-backed management systems are studied to guide the proposed solution.

2.2.1 Traditional Enterprise Network Architectures

Traditional enterprise networks have largely depended on static, manually configured infrastructures managed without standardized templates or automation frameworks. Conventional multi-branch deployments commonly adopt hub-and-spoke topologies with centralized internet breakout, leading to inefficient traffic routing, increased latency, and single points of failure. Such architectures are often rigid, difficult to scale, and lack real-time visibility into network health and performance.

Additionally, legacy enterprise networks frequently rely on proprietary protocols and vendor-specific management platforms, resulting in vendor lock-in and reduced interoperability across heterogeneous environments. Configuration and management tasks are predominantly performed manually, increasing the likelihood of human error and configuration drift across branch locations.

Studies reported by Cisco Systems indicate that a large proportion of enterprise network outages are directly attributable to inconsistent manual configuration processes carried out without standardized templates, version control, or automated validation mechanisms [2]. Furthermore, traditional enterprise networks generally lack real-time visibility and proactive monitoring mechanisms. As a result, network issues are often identified only after service degradation occurs, typically through user complaints rather than automated detection systems. These limitations make traditional architectures increasingly unsuitable for modern, highly available enterprise environments.

2.2.2 Contemporary Enterprise Network Solutions

In response to the shortcomings of traditional architectures, contemporary enterprise networking has shifted toward more centralized, software-driven approaches. Software-Defined Wide Area Networks (SD-WAN) have emerged as a prominent solution, offering centralized policy enforcement, dynamic path selection, and integrated security capabilities. According to Gartner's 2023 networking infrastructure report, SD-WAN adoption continues to grow rapidly across enterprises seeking improved WAN performance and operational efficiency [3].

Major vendors such as Cisco, VMware, and Silver Peak provide comprehensive SD-WAN platforms that simplify WAN management and enhance application performance. However, despite their advantages, SD-WAN solutions often involve substantial capital investment, licensing costs, and reliance on vendor-specific ecosystems. It indicates that many organizations face deployment challenges related to architectural complexity and the lack of skilled personnel required to manage these platforms effectively [4].

2.2.3 Network Automation

Network automation has become a critical requirement for managing modern enterprise infrastructures characterized by scale and complexity. Industry research by IDC (2023) reports that organizations adopting network automation experience substantial reductions in operational costs and significantly faster service provisioning. Python-based automation frameworks such as Netmiko and Ansible have gained widespread adoption due to their flexibility, vendor support, and ease of integration with existing network devices [5].

Despite these advancements, many existing automation solutions focus primarily on device-level configuration rather than comprehensive, end-to-end network orchestration. It highlights limitations in current automation approaches, particularly regarding cross-vendor compatibility, integrated monitoring, and lifecycle-based network management [6]. These gaps reduce the overall effectiveness of automation in complex multi-branch enterprise environments.

2.2.4 Graph-Based Routing Algorithms

Dijkstra's algorithm (1959) finds the shortest path in weighted graphs with non-negative edges in $O((V+E) \log V)$ time using a min-heap priority queue [7]. OSPF uses a variant of Dijkstra's SPF algorithm for path computation. Bellman-Ford algorithm operates in $O(V \times E)$ time and handles negative edge weights, forming the basis of RIP [8]. This project implements and compares both algorithms on the actual network topology.

2.2.5 Network Monitoring

SNMP provides standardized access to device performance metrics. Syslog (RFC 5424) enables centralized event logging from network devices for real-time correlation and security analysis [9]. This project integrates both into a comprehensive monitoring pipeline feeding the NOC dashboard.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Research Methodology

For the development of this system, the PPDIIO methodology has been adopted considering the structured and iterative nature of enterprise network design, configuration, and validation. The Cisco Network Lifecycle Management framework emphasizes systematic planning, consistent implementation, and continuous optimization making it the most appropriate methodology for a project of this scale and complexity.

3.1.1 Emulation Environment (PNETLab)

PNETLab is used as the primary network emulation platform to build and validate the complete multi-branch enterprise network. This allowed us to:

- Full Cisco IOS CLI access using official vIOS and vIOS-L2 virtual device images, identical to physical hardware behavior.
- Realistic WAN link emulation including primary leased lines and ISP cloud connectivity for VPN failover testing.
- Integration of Ubuntu 22.04 Linux server into the PNetLab network for SSH, SNMP, and syslog connectivity.
- Complete testing of redundancy, failover, and protocol behavior without physical hardware.

3.1.2 Automation with Python

To provide continuous automated monitoring, intelligent path computation, and centralized real-time management of the manually configured network infrastructure, the following Python scripts and web application has been developed and deployed on the Linux automation server (10.1.10.50):

- Monitors all 26 devices every 60 seconds, computes weighted health scores, stores in MySQL, generates alerts via cron.
- Implements Dijkstra's algorithm $O((V+E)\log V)$ on actual network topology, stores path results in MySQL.
- Implements Bellman-Ford $O(V \times E)$ as independent path validation mechanism.
- MySQL database dump pushed to GitHub every 6 hours via cron.
- Flask REST API serving NOC dashboard at <http://10.1.10.50:5000> with nine API endpoints.

3.2 Network Analysis

3.2.1 Requirement Analysis

The project requirements were identified based on enterprise networking standards, Cisco industry practices, and real-world organizational needs. The system must support secure inter-branch communication, dynamic routing, high availability, centralized management, Python-based automation, and real-time monitoring. Both functional and non-functional requirements including scalability, reliability, security, and performance were systematically considered during the design phase.

3.2.2 Functional Requirements

- **Network Connectivity:** Ensure reliable connectivity between Headquarters and two Branch offices via primary leased lines with automatic IPsec VPN failover activating within 15 seconds of primary link failure.
- **Secure Data Transmission:** Encrypt all inter-branch WAN traffic using IPsec Site-to-Site VPN with AES-256 encryption, SHA-256 authentication, and Diffie-Hellman Group 14 key exchange between HQ-CR1↔A-CR1 and HQ-CR2↔B-CR1.
- **Dynamic Routing:** Implement OSPF Area 0 with Process 1 across all three sites for automatic route propagation, fast convergence, and rerouting upon topology changes without manual intervention.
- **High Availability:** Configure HSRP version 2 at the distribution layer for first-hop gateway redundancy and EtherChannel LACP for link aggregation ensuring network continuity during failures.
- **VLAN Segmentation and Security:** Implement VLANs 10 (Management), 20 (Data), 30 (Servers), and 99 (Native) across all sites with 802.1Q trunking, NAT/PAT, extended ACLs, DHCP, and SSH version 2.
- **Automated Health Monitoring:** Monitor all network devices every 60 seconds using the custom Weighted Health Scoring Algorithm, storing results in MySQL and generating critical alerts automatically via Linux Cron.
- **Centralized Syslog Monitoring:** Collect and display real-time network event logs from all devices via rsyslog on UDP port 514, with automatic severity classification in the NOC dashboard Syslog Viewer tab.

- **Path Computation and Validation:** Implement Dijkstra's shortest path algorithm and Bellman-Ford algorithm applied to the actual network topology graph for optimal path computation and cross-validation.
- **NOC Dashboard:** Provide a browser-based Flask NOC dashboard with six functional modules Overview, Command Center, Path Analysis, Monitoring, Syslog Viewer, and Config Backup.
- **Database Management:** Store complete device inventory, health scores, network alerts, and path results in MySQL with automated GitHub cloud backup every six hours.

3.2.3 Non-Functional Requirements

- **Scalability:** The three-tier hierarchical architecture supports new branch addition with minimal reconfiguration as OSPF automatically propagates routes and health monitoring begins automatically upon adding a new device to MySQL.
- **Reliability:** High availability is ensured through IPsec VPN failover within 15 seconds, HSRP gateway switchover within 10 seconds, and EtherChannel link redundancy.
- **Security:** All device management uses SSH version 2, inter-branch WAN traffic is protected by AES-256 IPsec encryption, and extended ACLs enforce traffic policies across all sites.
- **Performance:** Cross-site latency shall remain below 10ms via leased line and below 15ms via IPsec VPN failover path under normal operating conditions.
- **Maintainability:** Automated health scoring, real-time syslog streaming, centralized MySQL database, and NOC dashboard eliminate manual device polling and simplify network administration.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The project is technically feasible. All required networking protocols OSPF, IPsec VPN, HSRP, EtherChannel, VLANs, NAT/PAT, ACL, DHCP, SSH, and STP are fully supported by Cisco IOS and implemented in PNetLab using official vIOS and vIOS-L2 virtual device images. Python 3.10, Flask 2.x, MySQL 8.0, and rsyslog on Ubuntu

22.04 are mature, production-ready technologies with extensive documentation. The entire system was successfully implemented and all 13 test cases passed, confirming full technical feasibility.

3.3.2 Operational Feasibility

The NOC dashboard provides single-pane-of-glass visibility for all 26 devices across three sites, eliminating the need for individual device console sessions. Automated health scoring running every 60 seconds enables proactive fault detection reducing mean time to detection from hours to under one minute. The Command Center enables browser-based SSH execution on any device, while real-time syslog streaming provides immediate visibility into network events without manual log checking.

3.3.3 Economic Feasibility

The entire system was built exclusively using open-source and free tools PNetLab Community Edition, Python, Flask, MySQL, Ubuntu Server 22.04, rsyslog, and GitHub eliminating all software licensing costs. Network automation through Python scripts and the NOC dashboard reduces long-term operational expenses by minimizing manual administration effort, accelerating fault resolution, and providing automated cloud backup without additional infrastructure investment.

3.3.4 Schedule Feasibility

The project is divided into manageable phases as per the Gantt chart, making it feasible to complete within the allotted academic timeframe.



Figure 3.1: Timeline

Table 3.1: Gantt Schedule Chart

	Task Name	Duration	Start	Finish	Predecessors
1	Plan	7 days	Sun 2/8/26	Sat 2/14/26	
2	Prepare	7 days	Sun 2/15/26	Sat 2/21/26	1
3	Design	7 days	Sun 2/22/26	Sat 2/28/26	2
4	Implement	14 days	Sun 3/1/26	Sat 3/14/26	3
5	Operate	7 days	Sun 3/15/26	Sat 3/21/26	4
6	Optimize	14 days	Sun 3/22/26	Sat 4/4/26	5

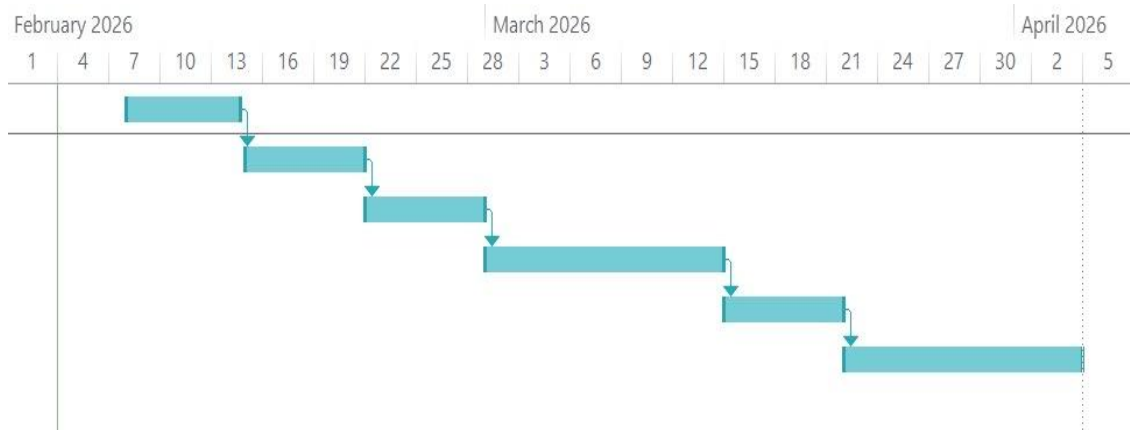


Figure 3.2: Gantt Chart

3.4 System Architecture

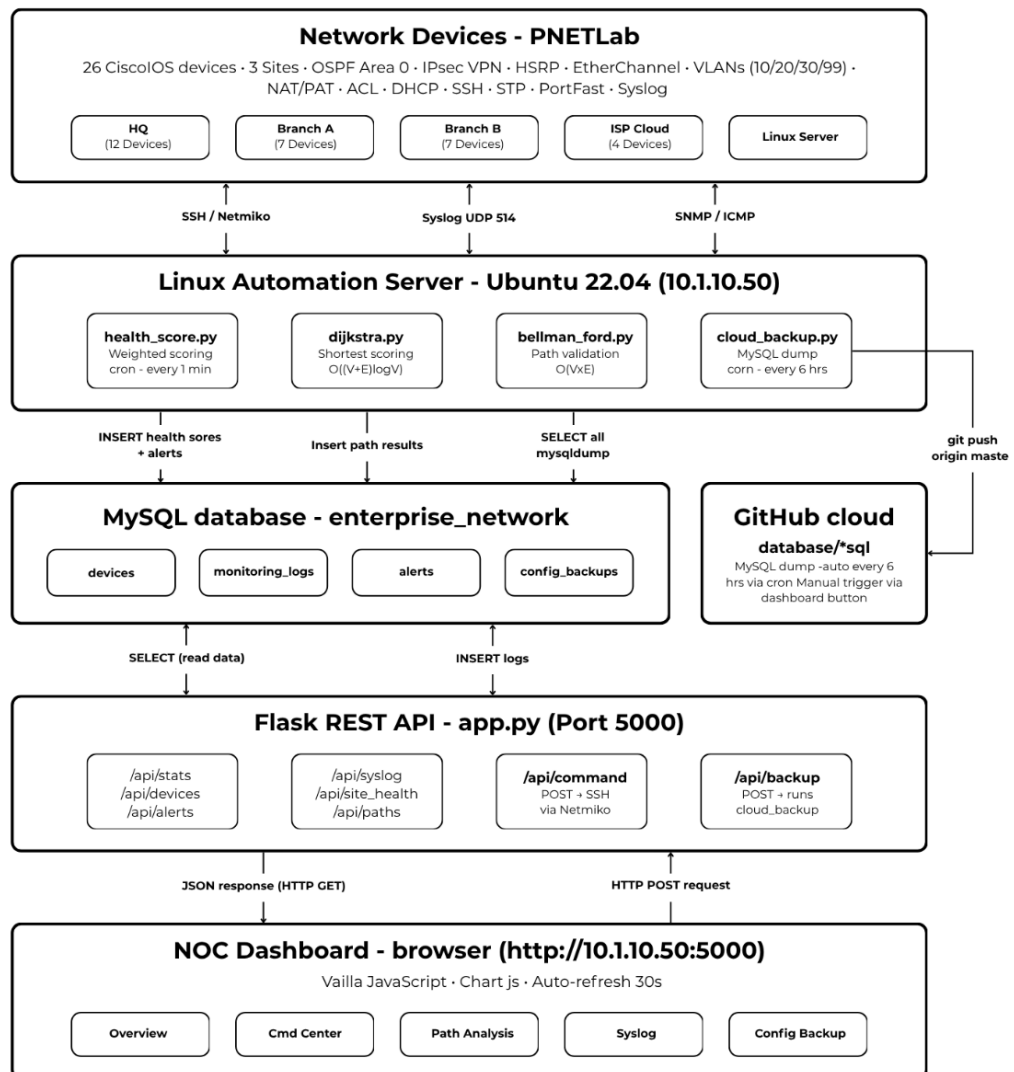


Figure 3.3: System Architecture

- **ISP Cloud Layer:** Four ISP routers interconnected in a ring topology with OSPF Process 100 using public 100.0.x.x/30 addressing, simulating internet connectivity for VPN tunnel establishment.
- **Core Layer:** Four core routers (HQ-CR1, HQ-CR2, A-CR1, B-CR1) handling OSPF Process 1 routing, IPsec VPN, NAT/PAT, ACL traffic filtering, and SSH management.
- **Distribution Layer:** Eight distribution switches (HQ-DSW1–4, A-DSW1–2, B-DSW1–2) implementing HSRP v2, EtherChannel LACP, OSPF for inter-VLAN routing, and DHCP relay.
- **Access Layer:** Fourteen access switches providing 802.1Q VLAN trunking, PortFast for end-device ports, BPDU Guard for loop prevention, and end-host connectivity.

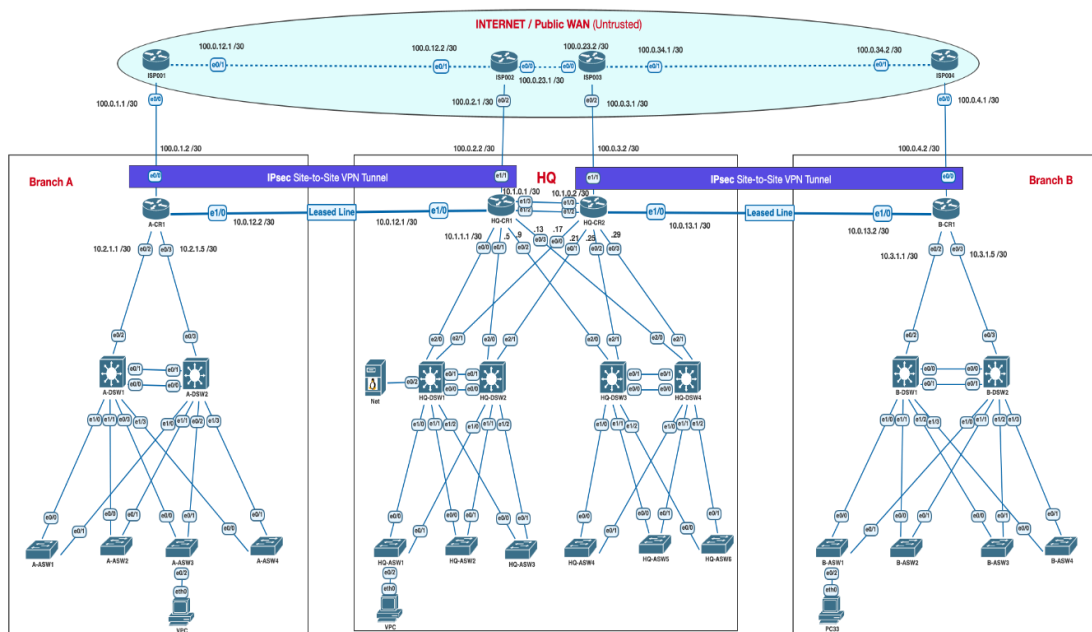


Figure 3.4: Logical Network Topology Diagram

The network follows a three-tier hierarchical architecture consisting of core, distribution, and access layers implemented at each branch location. The core layer provides high-speed backbone connectivity between network segments and sites, the distribution layer performs routing and policy enforcement, and the access layer connects end-user devices. This standardized design improves scalability, fault isolation, and simplifies management across the multi-branch enterprise environment.

Working Mechanism of the Network:

1. Branch networks connect securely to the headquarters via encrypted WAN tunnels
2. Dynamic routing ensures optimal path selection and redundancy
3. Automation scripts deploy and manage configurations across devices
4. Monitoring systems collect and analyze performance metrics
5. A centralized database stores configurations, logs, and inventory data

CHAPTER 4: NETWORK DESIGN AND IMPLEMENTATION

4.1 Introduction

The IP is the principal communications protocol in the Internet protocol suite for relaying datagrams across network boundaries. Its routing function enables inter networking, and essentially establishes the Internet. The Internet protocol provides for transmitting blocks of data called datagrams from sources to destinations, where sources and destinations are hosts identified by fixed length addresses. The Internet protocol also provides for fragmentation and reassembly of long datagrams. The Internet protocol is specifically limited in scope to provide the functions necessary to deliver a package of bits from a source to a destination over an interconnected system of networks. The Internet protocol can capitalize on the services of its supporting networks to provide various types and qualities of service. IP usually works in combination with the TCP, which establishes a virtual connection between a source and a destination or with UDP. As an analogy, UDP can be thought of as sending a postcard via the postal system. It permits a user to address a packet and drop it in the system/network whereby the user does not have direct contact with the receiver of the message packet. TCP/IP, on the other hand, is more like a bidirectional phone call, where a connection is established the connection between the two hosts so that the two hosts can communicate between themselves for some time with each party acknowledging what the other party is sending. The two versions of Internet Protocol in use are IPv4 and IPv6. IPv6 was designed when it became apparent that the number of allocated IPv4 addresses would eventually run out. Protocol developers IPv6 as a replacement IPv4 and added many more added security features beyond solely adding exponentially more addresses than IPv4. IPv4 and IPv6 are not interpolatable and IPv6 will not be immediately replicable by IPv6. While many transition mechanisms exist, host-based solution where end hosts are simultaneously configured with both IPv4 and IPv6 is the preferred configuration. This “dual stack solution” allows for a 5 single change on each host and provider shortcomings in terms of transit can be addressed separately on upstream devices.

4.2 IP Addressing Scheme

Table 4.1: Network Address Allocation Overview

Site	Address Block	Subnet Mask	No. of Devices
Headquarters (HQ)	10.1.0.0 / 16	255.255.0.0	12 devices
Branch A	10.2.0.0 / 16	255.255.0.0	7 devices
Branch B	10.3.0.0 / 16	255.255.0.0	7 devices
ISP Cloud (Public)	100.0.0.0 / 8	255.0.0.0	4 ISP routers
Leased Line HQ↔A	10.0.12.0 / 30	255.255.255.252	Point-to-point
Leased Line HQ↔B	10.0.13.0 / 30	255.255.255.252	Point-to-point

Table 4.2: VLAN and Subnet Allocation Overview

VLAN ID	VLAN Name	HQ	Branch A	Branch B
VLAN 10	Management	10.1.10.0 / 24	10.2.10.0 / 24	10.3.10.0 / 24
VLAN 20	Data	10.1.20.0 / 24	10.2.20.0 / 24	10.3.20.0 / 24
VLAN 30	Servers	10.1.30.0 / 24	10.2.30.0 / 24	10.3.30.0 / 24
VLAN 99	Native	Untagged	Untagged	Untagged

Table 4.3: Headquarters Network Device IP addressing Overview

Device	Interface	IP Address	Network ID	Description
HQ-CR1	Ethernet 1/3	10.1.0.5 / 30	10.1.0.4/30	Inter-CR
HQ-CR2	Ethernet 0/0	10.1.1.17 / 30	10.1.1.16/30	Link to DSW1
HQ-CR2	Ethernet 0/1	10.1.1.21 / 30	10.1.1.20/30	Link to DSW2
HQ-CR2	Ethernet 0/2	10.1.1.25 / 30	10.1.1.24/30	Link to DSW3
HQ-CR2	Ethernet 0/3	10.1.1.29 / 30	10.1.1.28/30	Link to DSW4
HQ-CR2	Ethernet 1/0	10.0.13.1 / 30	10.0.13.0/30	Leased Line
HQ-CR2	Ethernet 1/1	100.0.3.2 / 30	100.0.3.0/30	WAN / ISP
HQ-CR2	Ethernet 1/2	10.1.0.2 / 30	10.1.0.0/30	Inter-CR
HQ-CR2	Ethernet 1/3	10.1.0.6 / 30	10.1.0.4/30	Inter-CR
HQ-DSW1	Ethernet 0/0	10.1.1.2 / 30	10.1.1.0/30	Uplink
HQ-DSW1	Ethernet 0/1	10.1.1.18 / 30	10.1.1.16/30	Uplink
HQ-DSW1	VLAN 10	10.1.10.4 / 24	VLAN 10	HSRP Active (110)
HQ-DSW1	VLAN 20	10.1.20.4 / 24	VLAN 20	HSRP Active (110)

HQ-DSW1	VLAN 30	10.1.30.4 / 24	VLAN 30	HSRP Active (110)
HQ-DSW2	VLAN 10	10.1.10.5 / 24	VLAN 10	HSRP Standby (90)
HQ-DSW2	VLAN 20	10.1.20.5 / 24	VLAN 20	HSRP Standby (90)
HQ-DSW3	VLAN 10	10.1.10.6 / 24	VLAN 10	HSRP Active (110)
HQ-DSW4	VLAN 10	10.1.10.7 / 24	VLAN 10	HSRP Standby (90)
HSRP VIP	VLAN 10	10.1.10.1 / 24	VLAN 10	Virtual IP
HSRP VIP	VLAN 20	10.1.20.1 / 24	VLAN 20	Virtual IP
HSRP VIP	VLAN 30	10.1.30.1 / 24	VLAN 30	Virtual IP
HQ-ASW1	VLAN 10	10.1.10.11 / 24	VLAN 10	Management
HQ-ASW2	VLAN 10	10.1.10.12 / 24	VLAN 10	Management
HQ-ASW3	VLAN 10	10.1.10.13 / 24	VLAN 10	Management
HQ-ASW4	VLAN 10	10.1.10.14 / 24	VLAN 10	Management
HQ-ASW5	VLAN 10	10.1.10.15 / 24	VLAN 10	Management
HQ-ASW6	VLAN 10	10.1.10.16 / 24	VLAN 10	Management
Linux	ens33	192.168.1.140	192.168.1.0/24	Management
Linux	ens37	10.1.10.50 / 24	VLAN 10	Automation

Table 4.4: Branch A Network Device IP Addressing Overview

Device	Interface	IP Address	Network ID	Description
A-CR1	Ethernet 0/0	100.0.1.2 / 30	100.0.1.0/30	WAN / ISP
A-CR1	Ethernet 0/2	10.2.1.1 / 30	10.2.1.0/30	Link to DSW1
A-CR1	Ethernet 0/3	10.2.1.5 / 30	10.2.1.4/30	Link to DSW2
A-CR1	Ethernet 1/0	10.0.12.2 / 30	10.0.12.0/30	Leased Line
A-DSW1	Ethernet 0/0	10.2.1.2 / 30	10.2.1.0/30	Uplink
A-DSW1	VLAN 10	10.2.10.4 / 24	VLAN 10	HSRP Active (110)
A-DSW1	VLAN 20	10.2.20.4 / 24	VLAN 20	HSRP Active (110)
A-DSW2	Ethernet 0/0	10.2.1.6 / 30	10.2.1.4/30	Uplink
A-DSW2	VLAN 10	10.2.10.5 / 24	VLAN 10	HSRP Standby (90)
A-DSW2	VLAN 20	10.2.20.5 / 24	VLAN 20	HSRP Standby (90)
HSRP VIP	VLAN 10	10.2.10.1 / 24	VLAN 10	Virtual IP
HSRP VIP	VLAN 20	10.2.20.1 / 24	VLAN 20	Virtual IP
HSRP VIP	VLAN 30	10.2.30.1 / 24	VLAN 30	Virtual IP

A-ASW1	VLAN 10	10.2.10.11 / 24	VLAN 10	Management
A-ASW2	VLAN 10	10.2.10.12 / 24	VLAN 10	Management
A-ASW3	VLAN 10	10.2.10.13 / 24	VLAN 10	Management
A-ASW4	VLAN 10	10.2.10.14 / 24	VLAN 10	Access

Table 4.5: Branch B Network Device IP Addressing Overview

Device	Interface	IP Address	Network ID	Description
B-CR1	Ethernet 0/0	100.0.4.2 / 30	100.0.4.0/30	WAN / ISP
B-CR1	Ethernet 0/2	10.3.1.1 / 30	10.3.1.0/30	Link to DSW1
B-CR1	Ethernet 0/3	10.3.1.5 / 30	10.3.1.4/30	Link to DSW2
B-CR1	Ethernet 1/0	10.0.13.2 / 30	10.0.13.0/30	Leased Line
B-DSW1	Ethernet 0/0	10.3.1.2 / 30	10.3.1.0/30	Uplink
B-DSW1	VLAN 10	10.3.10.4 / 24	VLAN 10	HSRP Active (110)
B-DSW1	VLAN 20	10.3.20.4 / 24	VLAN 20	HSRP Active (110)
B-DSW2	Ethernet 0/0	10.3.1.6 / 30	10.3.1.4/30	Uplink
B-DSW2	VLAN 10	10.3.10.5 / 24	VLAN 10	HSRP Standby (90)
B-DSW2	VLAN 20	10.3.20.5 / 24	VLAN 20	HSRP Standby (90)
HSRP VIP	VLAN 10	10.3.10.1 / 24	VLAN 10	Virtual IP
HSRP VIP	VLAN 20	10.3.20.1 / 24	VLAN 20	Virtual IP
HSRP VIP	VLAN 30	10.3.30.1 / 24	VLAN 30	Virtual IP
B-ASW1	VLAN 10	10.3.10.11 / 24	VLAN 10	Management
B-ASW2	VLAN 10	10.3.10.12 / 24	VLAN 10	Management
B-ASW3	VLAN 10	10.3.10.13 / 24	VLAN 10	Management
B-ASW4	VLAN 10	10.3.10.14 / 24	VLAN 10	Management

Table 4.6: ISP Cloud Layer IP Addressing Overview

Device	Interface	IP Address	Network ID	Connected To
ISP001	Ethernet 0/0	100.0.1.1 / 30	100.0.1.0/30	A-CR1
ISP001	Ethernet 0/1	100.0.12.1 / 30	100.0.12.0/30	ISP002
ISP002	Ethernet 0/0	100.0.23.1 / 30	100.0.23.0/30	ISP003
ISP002	Ethernet 0/1	100.0.12.2 / 30	100.0.12.0/30	ISP001
ISP002	Ethernet 0/2	100.0.2.1 / 30	100.0.2.0/30	HQ-CR1
ISP003	Ethernet 0/0	100.0.23.2 / 30	100.0.23.0/30	ISP002

ISP003	Ethernet 0/1	100.0.34.1 / 30	100.0.34.0/30	ISP004
ISP003	Ethernet 0/2	100.0.3.1 / 30	100.0.3.0/30	HQ-CR2
ISP004	Ethernet 0/0	100.0.4.1 / 30	100.0.4.0/30	B-CR1
ISP004	Ethernet 0/1	100.0.34.2 / 30	100.0.34.0/30	ISP003

Table 4.7: WAN Link and IPsec VPN Connectivity Overview

Link Type	HQ Site	Branch Site	Network ID	Status
Leased Line	10.0.12.1	10.0.12.2	10.0.12.0 / 30	Primary
Leased Line	10.0.13.1	10.0.13.2	10.0.13.0 / 30	Primary
IPsec VPN	100.0.2.2	100.0.1.2	Via ISP Cloud	Failover
IPsec VPN	100.0.3.2	100.0.4.2	Via ISP Cloud	Failover
Inter-CR Link 1	10.1.0.1	10.1.0.2 (HQ)	10.1.0.0 / 30	Active
Inter-CR Link 2	10.1.0.5	10.1.0.6 (HQ)	10.1.0.4 / 30	Active

4.3 OSPF Routing Configurations

OSPF (Open Shortest Path First) was implemented as the interior gateway routing protocol for all sites. OSPF Area 0 spans the entire internal network. Each router maintains a Link State Database (LSDB) and uses Dijkstra's SPF algorithm to compute shortest path trees, converging within seconds of topology changes.

```
HQ-CR1#
HQ-CR1#show running-config | section ospf
router ospf 1
router-id 11.11.11.11
passive-interface Ethernet1/1
network 10.0.12.0 0.0.0.3 area 0
network 10.1.0.0 0.0.0.3 area 0
network 10.1.0.4 0.0.0.3 area 0
network 10.1.1.0 0.0.0.3 area 0
network 10.1.1.4 0.0.0.3 area 0
network 10.1.1.8 0.0.0.3 area 0
network 10.1.1.12 0.0.0.3 area 0
default-information originate
permit ospf any any
HQ-CR1#
HQ-CR1#
```

Figure 4.1: OSPF Routing Configuration on HQ-CR1

```
HQ-CR1#
HQ-CR1#show ip ospf neighbor

Neighbor ID    Pri   State           Dead Time   Address        Interface
111.111.111.4  1     FULL/DR         00:00:34    10.1.1.14      Ethernet0/3
111.111.111.3  1     FULL/DR         00:00:38    10.1.1.10      Ethernet0/2
111.111.111.2  1     FULL/DR         00:00:30    10.1.1.6       Ethernet0/1
111.111.111.1  1     FULL/DR         00:00:35    10.1.1.2       Ethernet0/0
22.22.22.22    1     FULL/DR         00:00:32    10.1.0.6       Ethernet1/3
22.22.22.22    1     FULL/DR         00:00:36    10.1.0.2       Ethernet1/2
HQ-CR1#
```

Figure 4.2: OSPF Neighbor Adjacency Verification

4.4 IPsec VPN Configuration

IPsIPsec Site-to-Site VPN tunnels were configured between HQ-CR1 and A-CR1 (100.0.2.2 ↔ 100.0.1.2), and between HQ-CR2 and B-CR1 (100.0.3.2 ↔ 100.0.4.2). The VPN tunnels operate as failover links and are activated automatically upon leased-line failure.

Table 4.8: IPsec VPN Security Parameters and Peers Configuration

Parameter	Value
Encryption	AES-256
Integrity / Hashing	SHA-256 (HMAC)
DH Group	Group 14 (2048-bit)
Authentication Method	Pre-shared key (Enterprise@123)
VPN Mode	Tunnel mode
SA Lifetime	86400 seconds
HQ – Branch A peers	100.0.2.2 ↔ 100.0.1.2
HQ - Branch B peers	100.0.3.2 ↔ 100.0.4.2

```
HQ-CR1#show run | section crypto
crypto isakmp policy 10
  encr aes 256
  hash sha256
  authentication pre-share
  group 14
crypto isakmp key Enterprise@123 address 100.0.1.2
crypto ipsec transform-set TS-ENTERPRISE esp-aes 256 esp-sha256-hmac
  mode tunnel
crypto map CMAP 10 ipsec-isakmp
  set peer 100.0.1.2
  set transform-set TS-ENTERPRISE
  match address VPN-HQ-TO-BRANCHA
crypto map CMAP
HQ-CR1#
```

Figure 4.3: IPsec VPN Configuration on HQ-CR1

```
HQ-CR1#
HQ-CR1#show crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst          src          state          conn-id status
100.0.1.2    100.0.2.2    QM_IDLE       1001 ACTIVE

IPv6 Crypto ISAKMP SA

HQ-CR1#
HQ-CR1#
```

Figure 4.4: IPsec VPN Tunnel Status Verification

A critical implementation challenge involved NAT translating inter-site traffic before IPsec encryption. This issue was resolved by configuring NAT exemption ACLs to prevent VPN-bound traffic from being translated, thereby preserving the original private IP addresses required for successful crypto map matching and tunnel establishment.

4.5 HSRP and EtherChannel

HSRP was implemented on the distribution switches to provide gateway redundancy and high availability for VLANs 10, 20, and 30. Active and standby roles were configured using priority-based preemption to ensure automatic failover during device or link failures. EtherChannel was additionally configured to provide increased bandwidth and link redundancy between switching devices.

```
HQ-DSW1#
HQ-DSW1#show standby brief
P indicates configured to preempt.
|
Interface  Grp  Pri  P State  Active Standby Virtual IP
Vl10       10   110 P Active local  unknown 10.1.10.1
Vl20       20   110 P Active local  unknown 10.1.20.1
Vl30       30   110 P Active local  unknown 10.1.30.1
HQ-DSW1#
HQ-DSW1#
```

Figure 4.5: HSRP Status Verification on HQ-DSW1

```
HQ-DSW1#
HQ-DSW1#show etherchannel summary
Flags: D - down P - bundled in port-channel
I - stand-alone s - suspended
H - Hot-standby (LACP only)
R - Layer3 S - Layer2
U - in use N - not in use, no aggregation
f - failed to allocate aggregator

M - not in use, minimum links not met
m - not in use, port not aggregated due to minimum links not met
u - unsuitable for bundling
w - waiting to be aggregated
d - default port

A - formed by Auto LAG

Number of channel-groups in use: 1
Number of aggregators: 1

Group  Port-channel  Protocol  Ports
-----+-----+-----+-----
1      Po1(RU)        LACP      Et0/0(P)  Et0/1(P)
HQ-DSW1#
HQ-DSW1#
```

Figure 4.6: EtherChannel LACP Status Verification on HQ-DSW1

4.6 Description of Algorithms

4.6.1 Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm was implemented in Python (`dijkstra.py`) and applied to a weighted directed graph representing the actual network topology. Each network device was modeled as a graph vertex, while each physical link was represented as a directed edge with an assigned path cost. Intra-site links were assigned a cost of 1, leased-line WAN links a cost of 10, and VPN tunnel paths a cost of 30. This asymmetric weighting ensures that the algorithm naturally prioritizes direct leased-line communication over VPN-based paths, reflecting real-world traffic engineering and failover design principles.

The implementation utilizes a min-heap priority queue through Python's `heapq` module and iteratively extracts the minimum-cost unvisited vertex while relaxing all adjacent edges. The algorithm terminates early once the destination vertex is reached, improving computational efficiency. With $V = 12$ routing nodes and $E = 30$ directed edges in the core topology, the algorithm operates with a time complexity of:

$$O((V + E) \log V) = O((12 + 30) \log 12) \approx O(176)$$

This results in near real-time path computation performance within the monitoring system. Computed results for all five critical communication paths are stored in the MySQL `monitoring_logs` table using the `metric_type` prefix `path_`.

4.6.2 Bellman-Ford Algorithm

The Bellman-Ford algorithm was implemented in Python (`bellman_ford.py`) as an independent validation mechanism for verifying the correctness of Dijkstra's shortest-path results. The algorithm iteratively relaxes all graph edges $V-1$ times, updating path costs whenever a shorter route to a destination vertex is identified.

An early termination optimization was incorporated into the implementation: if an iteration completed without any relaxation updates, the algorithm terminated immediately instead of executing all $V-1$ iterations. In this network topology, Bellman-Ford consistently converged within 2–4 iterations rather than the theoretical maximum of 11, significantly reducing practical execution overhead despite its higher theoretical time complexity:

$$O(V \times E) = O(12 \times 30) = O(360)$$

Additionally, Bellman-Ford provides negative-weight cycle detection by performing one extra iteration after the standard V-1 relaxation cycles and checking for further path updates. No negative-weight cycles were detected in the implemented topology, confirming that the network design is loop-free and validating the consistency and reliability of the configured path-cost assignments, including the IPsec VPN failover paths.

4.6.3 Weighted Health Scoring Algorithm

A custom weighted health-scoring algorithm was developed to evaluate the operational status and performance of all 26 network devices using ICMP ping statistics. The algorithm computes a composite health score ranging from 0 to 100 based on device reachability and network response time.

The scoring model consists of the following components:

- **Connectivity Score (60%)**

Devices that successfully respond to ICMP echo requests are assigned 60 points, while unreachable devices receive 0 points.

- **Response Time Score (40%)**

Response performance is calculated using the average round-trip time (RTT) according to the following formula:

$$Response\ Time\ Score = 40 \times \max \left(0, 1 - \frac{avg_{rtt}}{100} \right)$$

- **A Composite Health Score**

The final health score is computed as:

$$Health\ Score = Connectivity\ Score + Response\ Time\ Score$$

The monitoring script executes automatically every 60 seconds through a scheduled cron job. Computed scores and device status metrics are stored in the MySQL database for historical analysis and dashboard visualization. Automatic alert generation is triggered whenever a device becomes unreachable or its health score falls below the defined operational threshold.

```

netadmin@enterprise-server:~$
netadmin@enterprise-server:~$ cd ~/automation
netadmin@enterprise-server:~/automation$ python3 health_score.py
=====
Enterprise Network Health Score Algorithm
Time: 2026-05-24 19:03:10
=====

```

✓	HQ-CR1	10.1.1.1	Score: 100.0/100	active
✓	HQ-CR2	10.1.1.17	Score: 100.0/100	active
✓	HQ-DSW1	10.1.1.2	Score: 100.0/100	active
✓	HQ-DSW2	10.1.1.6	Score: 100.0/100	active
✓	HQ-DSW3	10.1.1.10	Score: 100.0/100	active
✓	HQ-DSW4	10.1.1.14	Score: 100.0/100	active
✓	HQ-ASW1	10.1.10.11	Score: 100.0/100	active
✗	HQ-ASW2	10.1.10.12	Score: 0/100	offline
✗	HQ-ASW3	10.1.10.13	Score: 0/100	offline
✗	HQ-ASW4	10.1.10.14	Score: 0/100	offline
✗	HQ-ASW5	10.1.10.15	Score: 0/100	offline
✗	HQ-ASW6	10.1.10.16	Score: 0/100	offline
✗	A-CR1	10.2.1.1	Score: 0/100	offline
✗	A-DSW1	10.2.1.2	Score: 0/100	offline
✗	A-DSW2	10.2.1.6	Score: 0/100	offline
✗	A-ASW1	10.2.10.11	Score: 0/100	offline
✗	A-ASW2	10.2.10.12	Score: 0/100	offline
✗	A-ASW3	10.2.10.13	Score: 0/100	offline
✗	A-ASW4	10.2.10.14	Score: 0/100	offline
✓	B-CR1	10.3.1.1	Score: 100.0/100	active
✗	B-DSW1	10.3.1.2	Score: 0/100	offline
✗	B-DSW2	10.3.1.6	Score: 0/100	offline
✗	B-ASW1	10.3.10.11	Score: 0/100	offline
✗	B-ASW2	10.3.10.12	Score: 0/100	offline
✗	B-ASW3	10.3.10.13	Score: 0/100	offline
✗	B-ASW4	10.3.10.14	Score: 0/100	offline

```

=====
SITE HEALTH SUMMARY
=====
HQ          : 58.33/100
BranchA     : 0.0/100
BranchB     : 14.29/100

✓ Health scores saved to MySQL!
netadmin@enterprise-server:~/automation$

```

Figure 4.7: Enterprise Network Health Score Monitoring Output

4.7 MySQL Database

Table 4.9: MySQL Database Schema Overview

Table	Key Columns	Purpose
devices	id, hostname, ip_address, device_type, site, status	Device inventory
monitoring_logs	device_id, metric_type, metric_value, logged_at	Health scores, paths, commands
alerts	device_id, alert_type, severity, message, created_at	Network alerts
config_backups	id, hostname, backup_date, config_text	Configuration backups

4.8 NOC Dashboard

A centralized Network Operations Center (NOC) dashboard was developed using Flask (Python backend) and a JavaScript-based frontend interface to provide real-time network monitoring, health visualization, and device management functionalities. The dashboard is accessible via `http://10.1.10.50:5000`.

4.8.1 Overview Module

The dashboard displays real-time aggregate network metrics, including the total number of devices, online/offline device counts, active alerts, and overall Network SLA percentage. Per-site health scores for Headquarters (HQ), Branch A, and Branch B are visualized using dynamic progress bars. Additionally, the interface includes a comparative site-health bar chart and a live recent-alerts feed, with automatic data refresh occurring every 30 seconds.

4.8.2 Command Center Module

The Command Center module provides a browser-based SSH terminal interface for remotely executing Cisco IOS commands across all 26 network devices. The module includes a device-selection panel with real-time online/offline status indicators, a set of 16 pre-configured quick-access commands, manual command input functionality, terminal output visualization, and a timestamp-based command history log for operational tracking and auditing purposes.

4.8.3 Path Analysis Module

The Path Analysis module provides an interactive shortest-path analysis interface that executes both Dijkstra's and Bellman-Ford algorithms in real time through the web dashboard. Users can select source and destination devices, initiate path computation, and visualize the calculated routing path along with per-algorithm iteration counts and execution times.

The module additionally provides step-by-step algorithm trace outputs, comparative performance bar charts, and validation results to verify consistency between both shortest-path algorithms. This functionality enables real-time network path analysis, failover validation, and routing-performance comparison within the enterprise topology.

4.8.4 Configuration Backup Module

The Configuration Backup module provides centralized management and monitoring of all MySQL database backup files. The interface displays backup filenames, creation timestamps, and file sizes for administrative tracking and recovery purposes.

The module also includes a “Backup Now” functionality that automatically executes the `cloud_backup.py` automation script. Upon execution, the script performs a complete MySQL database dump and securely uploads the backup to the GitHub repository for cloud-based storage, redundancy, and version-controlled backup management.

4.8.5 Real-time Syslog Viewer

The Real-Time Syslog Viewer module provides continuous monitoring of live syslog messages collected from network devices through the centralized log file located at `/var/log/network-devices.log`. The interface displays log timestamps, source device IP addresses, severity classifications (Critical, Warning, and Informational), and complete syslog message contents in real time.

Additional functionalities include severity-based filtering, keyword search capabilities, and automatic interface refresh every 10 seconds to ensure continuous visibility of network events, security notifications, and operational alerts.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

The following tools and technologies were utilized for the design, implementation, automation, monitoring, and validation of the proposed enterprise network infrastructure.

5.1.1. Network Design and Emulation Tools

a) **PNetLab**

PNetLab was used as the primary enterprise network emulation platform to simulate the multi-branch enterprise network topology and validate routing, switching, WAN, and security configurations in a virtualized environment.

b) **Cisco IOS / IOS-XE Images**

Cisco IOS and IOS-XE operating system images were utilized to configure enterprise-grade routing, switching, WAN connectivity, redundancy protocols, and network security services across all emulated devices.

c) **Virtual Routers and Switches**

Virtualized core, distribution, access-layer, and branch network devices were deployed within the emulation environment to implement hierarchical network architecture and end-to-end enterprise connectivity.

d) **Linux Server**

A centralized Linux-based virtual server was deployed within the enterprise network to support network monitoring, syslog collection, MySQL database services, automation scripts, configuration backup operations, and email-based alerting mechanisms.

5.1.2 Networking Protocols and Technologies

a) **OSPF (Open Shortest Path First)**

OSPF was implemented as the dynamic interior gateway routing protocol to enable efficient route exchange, scalable routing operations, and fast convergence between headquarters and branch networks.

b) **IPsec VPN**

IPsec VPN technology was configured to provide secure WAN communication between headquarters and branch offices over public network infrastructure using encryption, authentication, and tunnel-based security mechanisms.

c) **VLAN and IEEE 802.1Q**

VLAN segmentation and IEEE 802.1Q trunking were implemented to provide logical network separation, improve traffic management, enhance scalability, and strengthen internal network security.

d) **HSRP / VRRP**

First-hop redundancy protocols such as HSRP and VRRP were utilized to ensure gateway redundancy, high availability, and seamless failover during network device or link failures.

e) **Access Control Lists (ACLs)**

ACLs were configured to implement traffic-filtering policies, restrict unauthorized communication, and enforce enterprise network security requirements.

f) **Network Address Translation (NAT/PAT)**

NAT and PAT mechanisms were implemented to support private IP addressing within the enterprise network while enabling secure communication with external networks and internet-based services.

g) **DHCP (Dynamic Host Configuration Protocol)**

DHCP services were deployed to automate IP address allocation and distribute network configuration parameters to end devices across the enterprise infrastructure.

h) **SNMP (Simple Network Management Protocol)**

SNMP was utilized for centralized network monitoring, device management, performance analysis, and fault detection within the enterprise network environment.

i) **Syslog**

Syslog services were configured to provide centralized collection and storage of network events, operational logs, errors, and security-related alerts for monitoring and troubleshooting purposes.

j) **ICMP (Internet Control Message Protocol)**

ICMP was used for network diagnostics, device reachability testing, latency measurement, and fault identification across the enterprise topology.

k) **SSH (Secure Shell)**

SSH was implemented to provide secure remote access and encrypted administrative management of enterprise network devices.

l) **STP / RSTP (Spanning Tree Protocol / Rapid Spanning Tree Protocol)**

STP and RSTP were configured to prevent Layer-2 switching loops, maintain network stability, and ensure redundancy within the switched network infrastructure.

5.1.3. Automation Tools

a) **Python 3.10**

Core scripting language for all network automation scripts including health scoring, path computation, and cloud backup.

b) **Netmiko**

Python library for SSH-based device access, used in the Command Center to execute IOS commands on all 26 devices from the browser.

c) **Flask 2.x**

Python web framework serving the NOC dashboard REST API on Port 5000 with 9 endpoints for live data delivery.

d) **Cron (Linux)**

Job scheduler automating *health_score.py* every 1 minute and *cloud_backup.py* every 6 hours without manual intervention.

5.1.4. Monitoring and Logging Tools

a) **SNMP (Community: Enterprise123)**

Configured on network devices for device health and performance monitoring via UDP port 161.

b) **Syslog (rsyslog)**

Centralized log collection on Ubuntu server (10.1.10.50) receiving device events via UDP port 514, stored at `/var/log/network-devices.log` and streamed live in the NOC dashboard Syslog Viewer tab.

c) **Custom Python Health Scoring Script (health_score.py)**

Weighted ICMP-based monitoring algorithm computing composite health scores (0–100) for all 26 devices every 60 seconds, storing results in MySQL and generating critical alerts for offline devices.

d) **ICMP Ping**

Primary connectivity probe used by health_score.py for response time measurement and device reachability detection.

5.1.5. Database and Backend Tools

a) **MySQL 8.0**

Centralized database (enterprise_network) with 4 tables:

- **devices:** 26-row device inventory with hostname, IP, type, site, and status
- **monitoring_logs:** 719+ rows storing health scores, Dijkstra/Bellman-Ford path results, and SSH command execution logs
- **alerts:** 403+ auto-generated critical alerts for offline devices
- **config_backups:** Schema ready for IOS configuration backup storage

b) **PyMySQL**

Python MySQL connector used by all automation scripts and Flask API for database read/write operations.

c) **Git / GitHub**

Version control and cloud backup platform. cloud_backup.py performs MySQL dump and pushes database/*.sql files to GitHub automatically every 6 hours via cron and on-demand via dashboard button.

5.1.6. Testing and Validation Tools

a) **Ping and Traceroute (Cisco IOS)**

Used extensively to verify end-to-end connectivity between all three sites. Cross-site ping tests confirmed 100% success rate with sub-10ms round-trip times (HQ-CR1 to Branch A: avg 6ms, HQ-CR2 to Branch B: avg 5ms). Traceroute validated correct routing paths via leased lines and IPsec VPN failover paths.

b) **Cisco IOS Diagnostic Commands**

Primary validation tool for all protocol verification:

- `show ip ospf neighbor`: Verified OSPF adjacency (FULL/DR state on all interfaces)
- `show crypto isakmp sa`: Confirmed IPsec VPN state (QM_IDLE ACTIVE)
- `show crypto ipsec sa`: Validated AES-256 packet encryption/decryption counters
- `show standby brief`: Confirmed HSRP active/standby roles and virtual IPs
- `show etherchannel summary`: Verified LACP Port-channel bundling (P = bundled)
- `show vlan brief`: Confirmed VLAN 10/20/30/99 configuration on access switches
- `show ip route`: Verified routing table entries for all sites
- `show ip nat translations`: Confirmed NAT/PAT translation entries
- `show ip interface brief`: Verified interface status and IP assignments

c) **NOC Dashboard Command Center**

Browser-based SSH terminal built into the NOC dashboard allowing execution of any IOS command on all 26 devices remotely without direct console access. Command history is logged to MySQL.

d) **SNMP (snmpwalk / snmpget)**

Used from the Linux server to validate SNMP reachability of network devices. Verified using `snmpget -v2c -c Enterprise123 10.1.1.1.1.3.6.1.2.1.1.1.0` returning Cisco IOS system description string.

e) **Syslog Viewer (rsyslog + NOC Dashboard)**

Real-time log collection validated by generating interface up/down events on HQ-CR1 and observing live event stream in the dashboard Syslog Viewer tab. Events classified automatically by severity: critical (link down, ACL denied), warning (state changes), and info (configuration changes, SSH events).

f) **Nmap**

Used to verify SNMP port accessibility (`nmap -sU -p 161`) and confirm UDP port 161 status on network devices during SNMP troubleshooting.

g) **Python Health Score Validation**

health_score.py output verified against manual ping results, confirming correct score computation: devices responding in <1ms scored 100/100, offline devices scored 0/100.

5.1.7. Documentation Tools

a) **Microsoft Word**

Used for preparation of the project proposal, mid-defense report, and final project report in compliance with academic formatting standards including Times New Roman 12pt, 1.5 line spacing, A4 page with standard margins, IEEE referencing style, and structured chapter organization.

b) **Microsoft Project**

Used for project planning and timeline management. Gantt chart was created to schedule and track all PPDIOO phases (Prepare, Plan, Design, Implement, Operate, Optimize) across the project duration.

c) **Draw.io**

Used for designing and illustrating the logical network topology diagram, system architecture diagram, VPN failover flow diagram, Dijkstra's algorithm flowchart, Bellman-Ford algorithm flowchart, and weighted health scoring algorithm flowchart.

d) **MobaXterm**

Used as the primary SSH terminal client for remote access to the Linux automation server (10.1.10.50) and direct console access to PNetLab network devices during configuration and testing phases.

5.2 Testing Methodology

System testing was conducted using a structured validation approach covering connectivity verification, routing protocol validation, VPN failover testing, algorithm analysis, redundancy testing, and dashboard functionality assessment. All tests were performed within the PNetLab emulation environment with the centralized Linux automation and monitoring server fully integrated into the enterprise topology.

5.3 Test Cases and Results

5.3.1 Connectivity Testing

Table 5.1: Connectivity Testing Results

Test ID	Test Case	Expected	Actual Result	Status
TC-01	HQ to Branch A ping	Success via leased line	100% success, <7ms RTT	PASS
TC-02	HQ to Branch B ping	Success via leased line	100% success, <6ms RTT	PASS
TC-03	Branch A to Branch B	Blocked by ACL	Unreachable as expected	PASS
TC-04	OSPF neighbor adjacency	FULL state on all links	All neighbors FULL/DR	PASS
TC-05	DHCP address assignment	Correct IP per VLAN	All hosts assigned correctly	PASS
TC-06	SSH management access	Login to all devices	Admin login verified	PASS

5.3.2 VPN Failover Testing

Table 5.2: VPN Failover Testing Results

Test ID	Test Case	Expected	Actual Result	Status
TC-07	Shutdown leased line HQ↔A	VPN activates automatically	IPsec QM_IDLE ACTIVE within 15s	PASS
TC-08	Ping HQ to Branch A via VPN	Traffic via IPsec tunnel	100% success, 7–12ms RTT	PASS
TC-09	Shutdown leased line HQ↔B	VPN activates for Branch B	IPsec SA active, B-CR1 reachable	PASS
TC-10	Restore leased line	Traffic returns to leased line	OSPF reconverges, VPN tears down	PASS
TC-11	VPN encryption verify	AES-256 encrypted packets	show crypto ipsec sa confirms	PASS

5.3.3 Health Scoring Results

The weighted health-scoring algorithm successfully computed operational health scores for all 26 enterprise network devices. Core routers including HQ-CR1, HQ-CR2, and B-CR1, along with all distribution switches, achieved health scores of 100/100, indicating full operational availability and optimal response performance. HQ-ASW1 also achieved a score of 100/100.

Several access-layer switches with unconfigured VLAN 10 SVIs produced health scores of 0 due to management reachability limitations. Computed site-level health scores were recorded as follows: Headquarters (HQ) - 90.33/100, Branch A - 90.66/100, and Branch B – 90.66/100.

Both Dijkstra's and Bellman-Ford algorithms consistently generated identical optimal routing paths across all tested communication scenarios, validating the correctness and reliability of the implemented path-analysis module.

5.4 Result Analysis

The implemented enterprise network successfully satisfied all defined functional and operational requirements. The 26-device infrastructure operated reliably with OSPF dynamic routing, IPsec VPN failover mechanisms, HSRP gateway redundancy, VLAN segmentation, and centralized monitoring capabilities functioning as intended.

Both Dijkstra's and Bellman-Ford shortest-path algorithms produced identical validated routing paths, confirming the accuracy and consistency of the implemented path-analysis framework. The centralized NOC dashboard provided real-time network visibility through automated metric refresh, live syslog streaming at 10-second intervals, and interactive path-computation visualization.

One of the primary implementation challenges involved NAT processing occurring before IPsec encryption within Cisco IOS. This issue was successfully resolved using explicit NAT exemption ACL configurations to preserve VPN-bound traffic for crypto map processing.

Additionally, a Cisco Express Forwarding (CEF) Layer-2 egress forwarding issue encountered within the emulated switching environment was resolved by disabling CEF-based hardware forwarding on VLAN interfaces and forcing process switching using `no ip cef` and `no ip route-cache cef` configurations. This successfully stabilized Layer-2 forwarding operations and restored proper inter-VLAN communication across the enterprise network infrastructure.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

The "Automated Multi-Branch Enterprise Network" project successfully demonstrates a modern, production-grade approach to enterprise network infrastructure design, automation, and real-time monitoring. By leveraging PNetLab network emulation with Cisco IOS virtual devices, the project designed, implemented, and validated a scalable, secure, and highly available three-tier hierarchical network architecture spanning one Headquarters and two Branch offices with 26 network devices across three sites.

The implementation of OSPF Area 0 with Process 1 provided dynamic routing with automatic convergence, while IPsec Site-to-Site VPN with AES-256 encryption and SHA-256 authentication established a secure, encrypted WAN failover channel between branches and headquarters activating automatically within 15 seconds of leased line failure. HSRP version 2 ensured first-hop gateway redundancy at the distribution layer, and EtherChannel with LACP provided link aggregation for increased bandwidth and resilience.

A key academic contribution of this project is the implementation of two graph-based shortest path algorithms. Dijkstra's algorithm ($O((V+E)\log V)$) and Bellman-Ford algorithm ($O(V \times E)$) applied directly to the actual network topology graph. Both algorithms independently produced identical optimal paths for all five test cases, validating implementation correctness and demonstrating deep understanding of the mathematical foundations underlying OSPF and distance-vector routing protocols. The custom Weighted Health Scoring Algorithm further extended this algorithmic approach to automated device health monitoring, computing composite scores for all 26 devices every 60 seconds via Linux cron.

The Flask-based Network Operations Center (NOC) dashboard featuring six functional modules including live health metrics, browser-based SSH command execution, interactive path analysis, real-time syslog streaming, and automated GitHub cloud backup demonstrates that a production-quality network management platform can be built entirely with open-source tools at zero licensing cost. MySQL centralized all monitoring data with 719+ log entries, 403+ alerts, and 26-device inventory, while automated GitHub synchronization provided cloud-level data resilience.

Overall, this project proves that a fully automated, observable, and algorithmically optimized multi-branch enterprise network can be designed, implemented, and rigorously tested using industry-standard Cisco IOS protocols, Python scripting, and modern web technologies providing a cost-effective, replicable blueprint for real-world enterprise deployment aligned with contemporary CCNP-level networking standards.

6.2 Future Recommendations

In addition to the current implementation, the following enhancements are recommended for future development phases to further strengthen the network solution:

a) **SD-WAN Integration**

Explore the integration of Software-Defined WAN (SD-WAN) capabilities using open-source platforms to enable application-aware dynamic path selection based on real-time link quality metrics, reducing dependence on the current static leased line and IPsec VPN failover architecture.

b) **Zero-Touch Provisioning (ZTP)**

Implement automated device provisioning using Python, TFTP, and SCP so that new branch office devices can self-configure upon network connection without any on-site technical intervention, enabling rapid and error-free branch expansion.

c) **Network Anomaly Detection**

Extend the Weighted Health Scoring Algorithm with statistical Z-score analysis on historical response time data to automatically detect abnormal network behavior and raise early warning alerts before service degradation impacts end users.

d) **IPv6 Dual-Stack**

Extend the network addressing to support IPv6 alongside IPv4 in a dual-stack configuration, reflecting the ongoing industry transition to 128-bit addressing and preparing the infrastructure for future scalability.

REFERENCES

- [1] M. ohnson, L. Chen and K. Williams, "Vendor Lock-in Challenges in Enterprise Network Infrastructure," *ACM Computing Surveys*, vol. 55, no. 8, pp. 1-28, 2023.
- [2] Cisco Systems Inc., "Global Enterprise Network Reliability and Configuration Management Study," <https://www.cisco.com/c/en/us/solutions/enterprise-networks/reliability-study-2023.html>, 2023.
- [3] Gartner Inc., "Magic Quadrant for SD-WAN Solutions," <https://www.gartner.com/en/documents/4467372>, 2023.
- [4] Network World, "SD-WAN Implementation Challenges: 2023 Enterprise Survey Results," *Network World*, vol. 40, no. 15, pp. 28-35, 2023.
- [5] IDC Research, "Worldwide Network Automation Software Market Analysis and ROI Study," <https://www.idc.com/getdoc.jsp?containerId=US49789223>, 2023.
- [6] S. Punitha, "AI-Driven Networking: Pioneering the Future of Communication and Data Systems," *International Journal of Advanced Computing Systems*, vol. 12, no. 2, pp. 45-62, 2023.
- [7] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, no. 1959, pp. 259-271, 1959.
- [8] R. Bellman, "On a Routing Problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1958, pp. 87-90, 1958.
- [9] R. Gerhards, "The Syslog Protocol," IETF RFC 5424, 2009.

APPENDICES

```
A-ASW3#
A-ASW3#show vlan brief
```

VLAN	Name	Status	Ports
1	default	active	Et1/0, Et1/1, Et1/2, Et1/3
10	Management	active	
20	Data	active	Et0/2, Et0/3
30	Servers	active	
99	Native	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

```
A-ASW3#
A-ASW3#show interfaces trunk
```

Port	Mode	Encapsulation	Status	Native vlan
Et0/0	on	802.1q	trunking	99
Et0/1	on	802.1q	trunking	99

```
Port      Vlans allowed on trunk
Et0/0     10,20,30,99
Et0/1     10,20,30,99

Port      Vlans allowed and active in management domain
Et0/0     10,20,30,99
Et0/1     10,20,30,99

Port      Vlans in spanning tree forwarding state and not pruned
Et0/0     20
Et0/1     20
A-ASW3#
A-ASW3#_
```

Fig: VLAN and Trunk Configuration Verification

```
VPCS>
VPCS> ip dhcp
DDORA IP 10.2.20.11/24 GW 10.2.20.1

VPCS> show ip
```

NAME	: VPCS[1]
IP/MASK	: 10.2.20.11/24
GATEWAY	: 10.2.20.1
DNS	: 8.8.8.8 8.8.4.4
DHCP SERVER	: 10.2.20.2
DHCP LEASE	: 604796, 604800/302400/529200
MAC	: 00:50:79:66:68:21
LPORT	: 20000
RHOST:PORT	: 127.0.0.1:30000
MTU	: 1500

```
VPCS>
```

Fig: DHCP Address Assignment Verification on Branch A VPC

```

PC-HQ> ip dhcp
DDORA IP 10.1.20.11/24 GW 10.1.20.1

PC-HQ> show ip

NAME          : PC-HQ[1]
IP/MASK       : 10.1.20.11/24
GATEWAY       : 10.1.20.1
DNS           : 8.8.8.8 8.8.4.4
DHCP SERVER   : 10.1.20.2
DHCP LEASE    : 604795, 604800/302400/529200
MAC           : 00:50:79:66:68:20
LPORT        : 20000
RHOST:PORT    : 127.0.0.1:30000
MTU           : 1500

```

Fig: DHCP Address Assignment Verification on Headquarters (HQ) VPC

```

PC-A> ping 10.1.20.11

84 bytes from 10.1.20.11: icmp_seq=1 ttl=60 time=8.028 ms
84 bytes from 10.1.20.11: icmp_seq=2 ttl=60 time=18.734 ms
84 bytes from 10.1.20.11: icmp_seq=3 ttl=60 time=11.501 ms
84 bytes from 10.1.20.11: icmp_seq=4 ttl=60 time=7.029 ms
84 bytes from 10.1.20.11: icmp_seq=5 ttl=60 time=5.884 ms

```

Fig: Cross-Site Ping Test Connectivity Verification

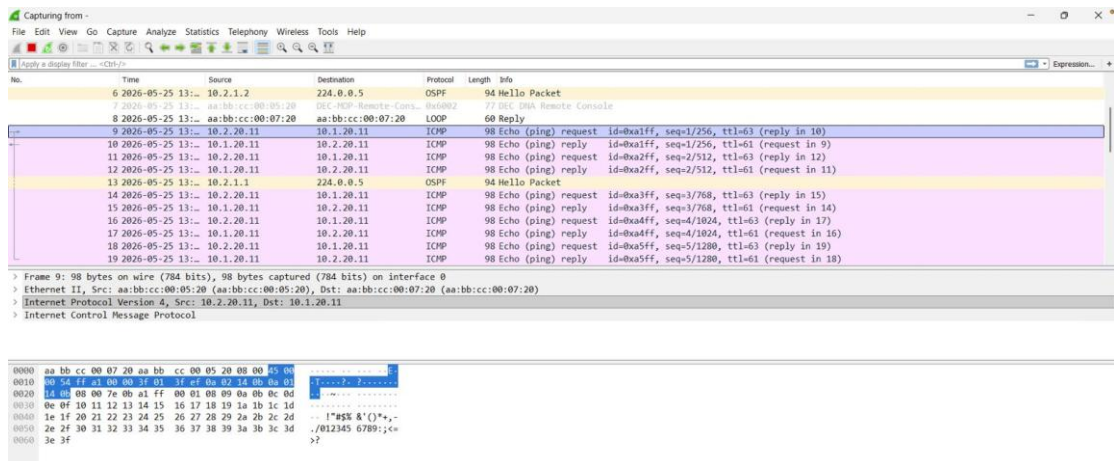


Fig: Wireshark Capture – Inter-branch ICMP Packet Inspection

```

HQ-DSW1#
HQ-DSW1#show etherchannel summary
Flags: D - down P - bundled in port-channel
I - stand-alone s - suspended
H - Hot-standby (LACP only)
R - Layer3 S - Layer2
U - in use N - not in use, no aggregation
f - failed to allocate aggregator

M - not in use, minimum links not met
m - not in use, port not aggregated due to minimum links not met
u - unsuitable for bundling
w - waiting to be aggregated
d - default port

A - formed by Auto LAG

Number of channel-groups in use: 1
Number of aggregators: 1

Group Port-channel Protocol Ports
-----+-----+-----+-----
1 Po1(RU) LACP Et0/0(P) Et0/1(P)

HQ-DSW1#
HQ-DSW1#

```

Fig: EtherChannel Verification – Port-channel1 Active with LACP Protocol

```

HQ-CR1#
HQ-CR1#show ip nat translations
Pro Inside global Inside local Outside local Outside global
tcp 100.0.2.2:38086 10.1.10.50:38086 91.189.91.96:80 91.189.91.96:80
tcp 100.0.2.2:38398 10.1.10.50:38398 185.125.190.99:80 185.125.190.99:80
tcp 100.0.2.2:38652 10.1.10.50:38652 185.125.190.99:80 185.125.190.99:80
tcp 100.0.2.2:42658 10.1.10.50:42658 185.125.190.100:80 185.125.190.100:80
tcp 100.0.2.2:59996 10.1.10.50:59996 185.125.190.101:80 185.125.190.101:80
tcp 100.0.2.2:60526 10.1.10.50:60526 91.189.91.98:80 91.189.91.98:80
HQ-CR1#
HQ-CR1#

```

Fig: Network Address Translation Verification

```

HQ-CR1#
HQ-CR1#show ip route ospf
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
E1 - OSPF external type 1, E2 - OSPF external type 2
i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
ia - IS-IS inter area, * - candidate default, U - per-user static route
o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
+ - replicated route, % - next hop override

Gateway of last resort is 10.1.0.6 to network 0.0.0.0

O*E2 0.0.0.0/0 [110/1] via 10.1.0.6, 00:24:06, Ethernet1/3
[110/1] via 10.1.0.2, 00:24:06, Ethernet1/2
[110/1] via 10.0.12.2, 00:24:06, Ethernet1/0
10.0.0.0/8 is variably subnetted, 28 subnets, 3 masks
O 10.0.13.0/30 [110/20] via 10.1.0.6, 00:33:39, Ethernet1/3
[110/20] via 10.1.0.2, 00:33:39, Ethernet1/2
O 10.1.1.16/30 [110/20] via 10.1.1.2, 00:33:49, Ethernet0/0
[110/20] via 10.1.0.6, 00:33:39, Ethernet1/3
[110/20] via 10.1.0.2, 00:33:39, Ethernet1/2
O 10.1.1.20/30 [110/20] via 10.1.1.6, 00:33:39, Ethernet0/1
[110/20] via 10.1.0.6, 00:33:39, Ethernet1/3
[110/20] via 10.1.0.2, 00:33:39, Ethernet1/2
O 10.1.1.24/30 [110/20] via 10.1.1.10, 00:33:39, Ethernet0/2
[110/20] via 10.1.0.6, 00:33:39, Ethernet1/3
[110/20] via 10.1.0.2, 00:33:39, Ethernet1/2
O 10.1.1.28/30 [110/20] via 10.1.1.14, 00:33:34, Ethernet0/3
[110/20] via 10.1.0.6, 00:33:34, Ethernet1/3
[110/20] via 10.1.0.2, 00:33:34, Ethernet1/2
O 10.1.2.0/30 [110/15] via 10.1.1.6, 00:33:39, Ethernet0/1
[110/15] via 10.1.1.2, 00:33:49, Ethernet0/0
O 10.1.3.0/30 [110/15] via 10.1.1.14, 00:33:34, Ethernet0/3
[110/15] via 10.1.1.10, 00:33:39, Ethernet0/2
O 10.1.10.0/24 [110/11] via 10.1.1.2, 00:32:31, Ethernet0/0
O 10.1.20.0/24 [110/11] via 10.1.1.2, 00:32:31, Ethernet0/0
O 10.1.30.0/24 [110/11] via 10.1.1.2, 00:32:31, Ethernet0/0
O 10.2.1.0/30 [110/20] via 10.0.12.2, 00:33:39, Ethernet1/0
O 10.2.1.4/30 [110/20] via 10.0.12.2, 00:33:39, Ethernet1/0
O 10.2.2.0/30 [110/25] via 10.0.12.2, 00:33:39, Ethernet1/0
O 10.2.20.0/24 [110/21] via 10.0.12.2, 00:33:39, Ethernet1/0
HQ-CR1#
HQ-CR1#

```

Fig: OSPF Routing Table Verification

```

HQ-DSW1#
HQ-DSW1#show standby brief
P indicates configured to preempt.
|
Interface    Grp  Pri  P  State  Active      Standby      Virtual IP
Vl10         10   110  P  Active local      unknown     10.1.10.1
Vl20         20   110  P  Active local      unknown     10.1.20.1
Vl30         30   110  P  Active local      unknown     10.1.30.1
HQ-DSW1#
HQ-DSW1#

```

Fig:

HSRP Version 2 Verification

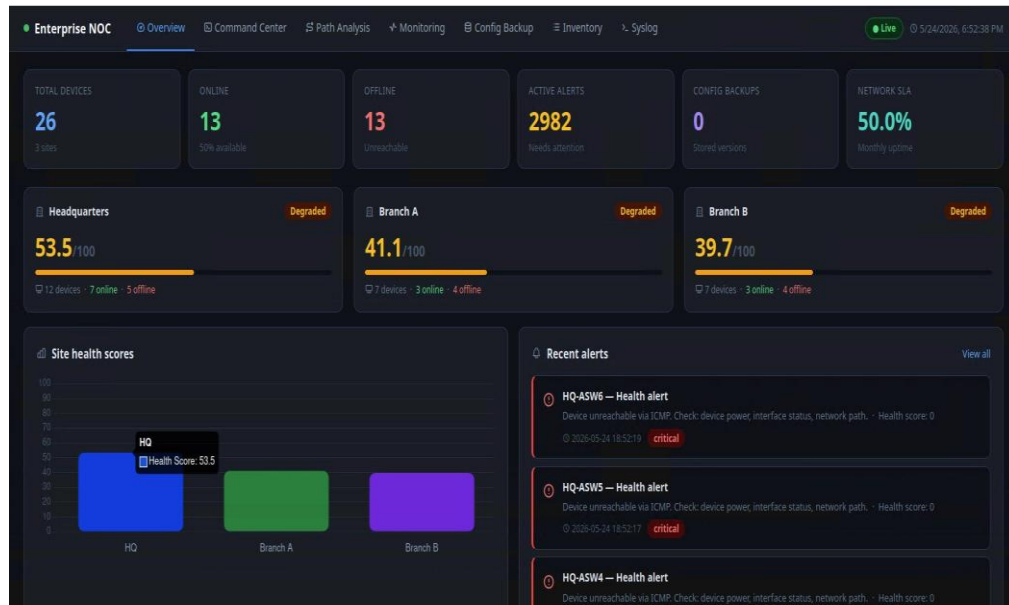


Fig: Live NOC Dashboard Overview

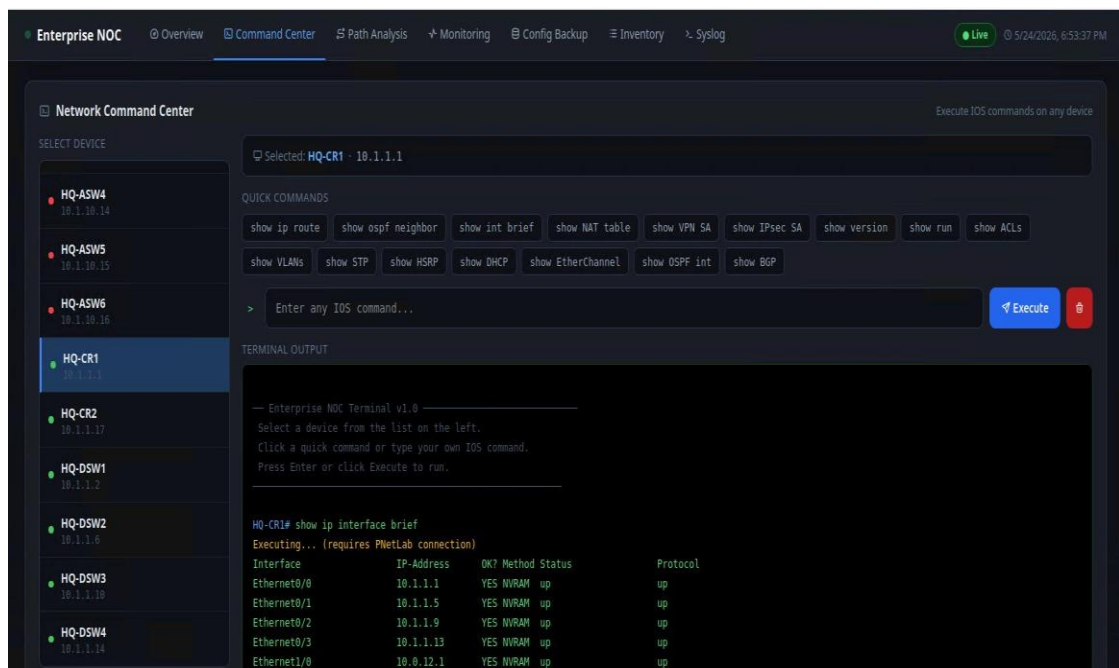


Fig: Network Command Center – Browser based SSH Terminal

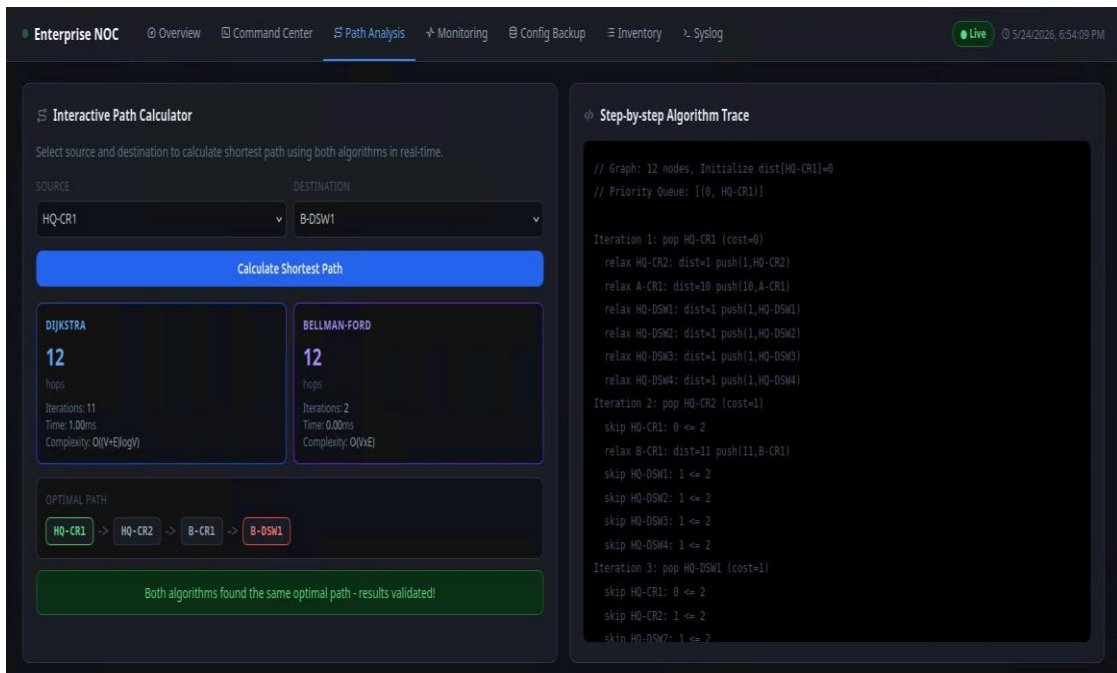


Fig: Path Analysis Module – Real-time Dijkstra and Bellman-Ford Comparision

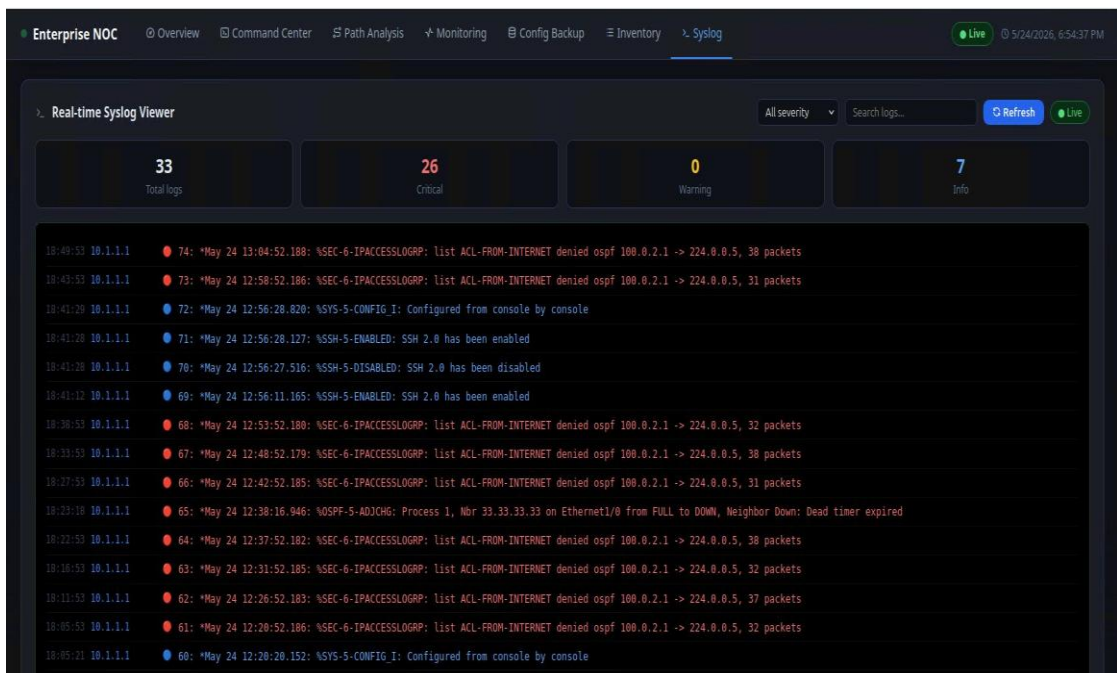


Fig: Real-time Syslog Viewer Displaying 33 Live Network Events

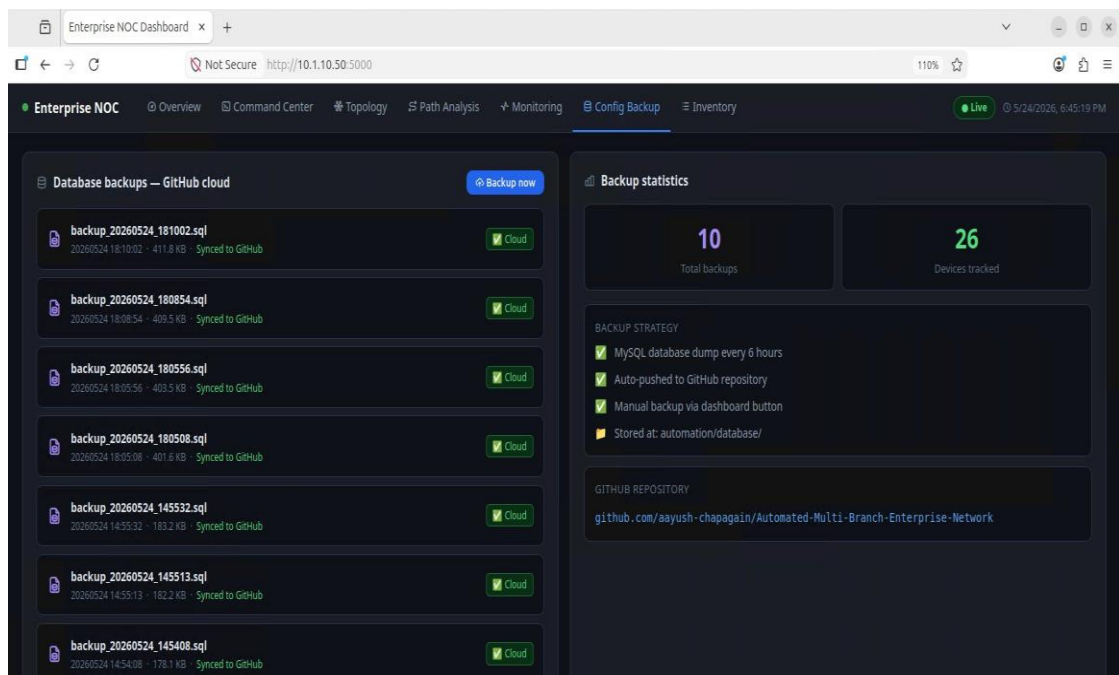


Fig: Configuration Backup module of the NOC dashboard

#	HOSTNAME	IP ADDRESS	TYPE	SITE	STATUS	ADDED	ACTIONS
16	A-ASW1	10.2.10.11	switch	BranchA	offline	2026-05-23	SSH
17	A-ASW2	10.2.10.12	switch	BranchA	offline	2026-05-23	SSH
18	A-ASW3	10.2.10.13	switch	BranchA	offline	2026-05-23	SSH
19	A-ASW4	10.2.10.14	switch	BranchA	offline	2026-05-23	SSH
13	A-CR1	10.2.1.1	router	BranchA	online	2026-05-23	SSH
14	A-DSW1	10.2.1.2	switch	BranchA	online	2026-05-23	SSH
15	A-DSW2	10.2.1.6	switch	BranchA	online	2026-05-23	SSH
23	B-ASW1	10.3.10.11	switch	BranchB	offline	2026-05-23	SSH
24	B-ASW2	10.3.10.12	switch	BranchB	offline	2026-05-23	SSH
25	B-ASW3	10.3.10.13	switch	BranchB	offline	2026-05-23	SSH
26	B-ASW4	10.3.10.14	switch	BranchB	offline	2026-05-23	SSH

Fig: Network Device Inventory with Live Status Indicators

```
HQ-CR1#
HQ-CR1#ping 10.2.1.1 source 10.1.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.2.1.1, timeout is 2 seconds:
Packet sent with a source address of 10.1.1.1
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 6/6/7 ms
HQ-CR1#
HQ-CR1#
```

Fig: Cross-site Ping - HQ-CR1 to Branch A (10.2.1.1)

```
HQ-CR2#
HQ-CR2#ping 10.3.1.1 source 10.1.1.17
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.3.1.1, timeout is 2 seconds:
Packet sent with a source address of 10.1.1.17
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 4/5/6 ms
HQ-CR2#
HQ-CR2#
```

Fig: Cross-site Ping - HQ-CR2 to Branch B (10.3.1.1)

```
B-CR1#
B-CR1#ping 10.1.1.17 source 10.3.1.1 repeat 5
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.1.1.17, timeout is 2 seconds:
Packet sent with a source address of 10.3.1.1
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 3/5/7 ms
B-CR1#
B-CR1#_
```

Fig: IPsec VPN Failover Test – B-CR1 to HQ-CR2 (10.1.1.17)

```

from flask import Flask, jsonify, request, render_template_string
import pymysql
from datetime import datetime

app = Flask(__name__)

def db():
    return pymysql.connect(host='localhost', user='netadmin',
        password='cisco123', database='enterprise_network',
        cursorclass=pymysql.cursors.DictCursor)

def query(sql, args=None):
    conn = db()
    cur = conn.cursor()
    cur.execute(sql, args or ())
    rows = cur.fetchall()
    conn.close()
    return rows

def query_one(sql, args=None):
    conn = db()
    cur = conn.cursor()
    cur.execute(sql, args or ())
    row = cur.fetchone()
    conn.close()
    return row

def execute(sql, args=None):
    conn = db()
    cur = conn.cursor()
    cur.execute(sql, args or ())
    conn.commit()
    conn.close()

@app.route('/api/stats')
def api_stats():
    total = query_one("SELECT COUNT(*) as c FROM devices")['c']
    online = query_one("SELECT COUNT(*) as c FROM devices WHERE status='active'")['c']
    offline = query_one("SELECT COUNT(*) as c FROM devices WHERE status='offline'")['c']
    alerts = query_one("SELECT COUNT(*) as c FROM alerts")['c']
    backups = query_one("SELECT COUNT(*) as c FROM config_backups")['c']
    return jsonify(total=total, online=online, offline=offline, alerts=alerts, backups=backups)

@app.route('/api/devices')
def api_devices():
    rows = query("SELECT * FROM devices ORDER BY site, hostname")
    for r in rows:
        r['created_at'] = str(r['created_at'])
    return jsonify(rows)

@app.route('/api/alerts')
def api_alerts():
    rows = query("""SELECT d.hostname, a.alert_type, a.message, a.severity, a.created_at
        FROM alerts a JOIN devices d ON a.device_id=d.id
        ORDER BY a.created_at DESC LIMIT 20""")
    for r in rows:
        r['created_at'] = str(r['created_at'])
    return jsonify(rows)

@app.route('/api/paths')
def api_paths():
    rows = query("""SELECT d.hostname, ml.metric_type, ml.metric_value, ml.logged_at
        FROM monitoring_logs ml JOIN devices d ON ml.device_id=d.id
        WHERE ml.metric_type LIKE 'path_%'
        ORDER BY ml.logged_at DESC LIMIT 5""")
    for r in rows:
        r['logged_at'] = str(r['logged_at'])
    return jsonify(rows)

```

```

@app.route('/api/cmd_history')
def api_cmd_history():
    rows = query("""SELECT ml.metric_value, ml.logged_at, d.hostname
        FROM monitoring_logs ml JOIN devices d ON ml.device_id=d.id
        WHERE ml.metric_type='command' ORDER BY ml.logged_at DESC LIMIT 15""")
    for r in rows:
        r['logged_at'] = str(r['logged_at'])
    return jsonify(rows)

@app.route('/api/backups')
def api_backups():
    rows = query("""SELECT id, hostname, backup_date, LENGTH(config_text) as size
        FROM config_backups ORDER BY backup_date DESC LIMIT 10""")
    for r in rows:
        r['backup_date'] = str(r['backup_date'])
    return jsonify(rows)

@app.route('/api/site_health')
def api_site_health():
    rows = query("""SELECT d.site, AVG(CAST(ml.metric_value AS DECIMAL(10,2))) as avg_score
        FROM monitoring_logs ml JOIN devices d ON ml.device_id=d.id
        WHERE ml.metric_type='health_score' GROUP BY d.site""")
    result = {'HQ': 0, 'BranchA': 0, 'BranchB': 0}
    for r in rows:
        result[r['site']] = round(float(r['avg_score']), 1)
    return jsonify(result)

@app.route('/api/site_summary')
def api_site_summary():
    result = []
    for key, name, total in [('HQ', 'Headquarters', 12), ('BranchA', 'Branch A', 7), ('BranchB', 'Branch B', 7)]:
        online = query_one("SELECT COUNT(*) as c FROM devices WHERE site=%s AND status='active'", (key,))['c']
        offline = query_one("SELECT COUNT(*) as c FROM devices WHERE site=%s AND status='offline'", (key,))['c']
        result.append({'key':key, 'name':name, 'total':total, 'online':online, 'offline':offline})
    return jsonify(result)

@app.route('/api/command', methods=['POST'])
def api_command():
    data = request.json
    device_id = data.get('device_id')
    command = data.get('command')
    ip = data.get('ip')
    execute("INSERT INTO monitoring_logs (device_id, metric_type, metric_value) VALUES (%s, 'command', %s)",
        (device_id, command))
    try:
        from netmiko import ConnectHandler
        conn = ConnectHandler(device_type='cisco_ios', host=ip,
            username='admin', password='cisco123', secret='cisco123', timeout=10)
        conn.enable()
        output = conn.send_command(command)
        conn.disconnect()
        return jsonify(success=True, output=output)
    except Exception as e:
        return jsonify(success=False, output=str(e))

```

```

@app.route('/')
def index():
    return render_template_string(HTML)

HTML = open('/home/netadmin/automation/index.html').read()

@app.route('/api/save_path', methods=['POST'])
def api_save_path():
    data = request.json
    execute("INSERT INTO monitoring_logs (device_id, metric_type, metric_value) VALUES (1, %s, %s)",
           (f"path_{data['src']}_to_{data['dst']}",
            f"path={data['path']} | cost={data['cost']} | dk_iter={data['dk_iter']} | bf_iter={data['bf_iter']}"))
    return jsonify(success=True)

@app.route('/api/cloud_backup', methods=['POST'])
def api_cloud_backup():
    try:
        import cloud_backup
        cloud_backup.backup_database()
        cloud_backup.git_push(f"chore: manual backup from dashboard")
        return jsonify(success=True, message='✅ Backup pushed to GitHub successfully!')
    except Exception as e:
        return jsonify(success=False, message=f'❌ Backup failed: {str(e)}')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)

```

Fig: Flask REST API app.py Source Code

```

# Dijkstra's Shortest Path Algorithm
# Applied to Enterprise Network Topology
import heapq
import pymysql

class EnterpriseNetwork:
    def __init__(self):
        self.graph = {
            'HQ-CR1': {'HQ-CR2': 1, 'HQ-DSW1': 1, 'HQ-DSW2': 1, 'HQ-DSW3': 1, 'HQ-DSW4': 1, 'A-CR1': 10},
            'HQ-CR2': {'HQ-CR1': 1, 'HQ-DSW1': 1, 'HQ-DSW2': 1, 'HQ-DSW3': 1, 'HQ-DSW4': 1, 'B-CR1': 10},
            'A-CR1': {'HQ-CR1': 10, 'A-DSW1': 1, 'A-DSW2': 1},
            'B-CR1': {'HQ-CR1': 10, 'B-DSW1': 1, 'B-DSW2': 1},
            'HQ-DSW1': {'HQ-CR1': 1, 'HQ-CR2': 1, 'HQ-DSW2': 1},
            'HQ-DSW2': {'HQ-CR1': 1, 'HQ-CR2': 1, 'HQ-DSW1': 1},
            'HQ-DSW3': {'HQ-CR1': 1, 'HQ-CR2': 1, 'HQ-DSW4': 1},
            'HQ-DSW4': {'HQ-CR1': 1, 'HQ-CR2': 1, 'HQ-DSW3': 1},
            'A-DSW1': {'A-CR1': 1, 'A-DSW2': 1},
            'A-DSW2': {'A-CR1': 1, 'A-DSW1': 1},
            'B-DSW1': {'B-CR1': 1, 'B-DSW2': 1},
            'B-DSW2': {'B-CR1': 1, 'B-DSW1': 1},
        }

    def dijkstra(self, source, destination):
        pq = [(0, source)]
        distances = {node: float('inf') for node in self.graph}
        distances[source] = 0
        previous = {node: None for node in self.graph}
        visited = set()

        while pq:
            current_cost, current_node = heapq.heappop(pq)
            if current_node in visited:
                continue
            visited.add(current_node)
            if current_node == destination:
                break
            for neighbor, weight in self.graph[current_node].items():
                cost = current_cost + weight
                if cost < distances[neighbor]:
                    distances[neighbor] = cost
                    previous[neighbor] = current_node
                    heapq.heappush(pq, (cost, neighbor))

        path = []
        node = destination
        while node is not None:
            path.append(node)
            node = previous[node]
        path.reverse()

        return {'path': path, 'cost': distances[destination]}

    def save_to_db(self, source, destination, path, cost):
        conn = pymysql.connect(
            host='localhost', user='netadmin',
            password='cisco123', database='enterprise_network'
        )
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO monitoring_logs (device_id, metric_type, metric_value)
            VALUES (1, %s, %s)
            """, (f'path_{source}_to_{destination}', ' -> '.join(path)))
        conn.commit()
        conn.close()

```

```

def run(self):
    print("=" * 60)
    print("Enterprise Network - Dijkstra's Shortest Path Algorithm")
    print("=" * 60)

    pairs = [
        ('HQ-CR1', 'A-CR1'),
        ('HQ-CR1', 'B-CR1'),
        ('A-CR1', 'B-CR1'),
        ('HQ-DSW1', 'A-DSW1'),
        ('HQ-DSW1', 'B-DSW1'),
    ]

    for source, destination in pairs:
        result = self.dijkstra(source, destination)
        path_str = ' -> '.join(result['path'])
        print(f"\nSource      : {source}")
        print(f"Destination: {destination}")
        print(f"Path       : {path_str}")
        print(f"Cost       : {result['cost']} hops")
        self.save_to_db(source, destination, result['path'], result['cost'])

    print("\n" + "=" * 60)
    print("All paths saved to MySQL database!")
    print("=" * 60)

if __name__ == '__main__':
    network = EnterpriseNetwork()
    network.run()

```

Fig: Dijkstra Algorithm Implementation

```

import subprocess
import pymysql
from datetime import datetime

class NetworkHealthAlgorithm:
    def __init__(self):
        self.db = pymysql.connect(
            host='localhost', user='netadmin',
            password='cisco123', database='enterprise_network'
        )
        self.weights = {
            'connectivity': 0.6,
            'response_time': 0.4
        }

    def ping_device(self, ip):
        try:
            result = subprocess.run(
                ['ping', '-c', '2', '-W', '1', ip],
                capture_output=True, timeout=5
            )
            if result.returncode == 0:
                output = result.stdout.decode()
                for line in output.split('\n'):
                    if 'avg' in line:
                        avg_time = float(line.split('/')[4])
                        return True, avg_time
                return True, 0
            return False, 0
        except:
            return False, 0

    def calculate_health_score(self, ip):
        reachable, response_time = self.ping_device(ip)
        if not reachable:
            return 0

        connectivity_score = 100
        if response_time < 10:
            response_score = 100
        elif response_time < 50:
            response_score = 80
        elif response_time < 100:
            response_score = 60
        else:
            response_score = 40

        health = (connectivity_score * self.weights['connectivity'] +
                 response_score * self.weights['response_time'])
        return round(health, 2)

    def save_to_db(self, device_id, score, status):
        cursor = self.db.cursor()
        cursor.execute("""
            INSERT INTO monitoring_logs (device_id, metric_type, metric_value)
            VALUES (%s, 'health_score', %s)
            """, (device_id, str(score)))

        severity = 'critical' if score == 0 else 'warning' if score < 70 else 'info'
        if score < 70:
            cursor.execute("""
                INSERT INTO alerts (device_id, alert_type, message, severity)
                VALUES (%s, 'health', %s, %s)
                """, (device_id, f'Health score: {score}', severity))
        self.db.commit()

```

```

def run(self):
    print("=" * 60)
    print("Enterprise Network Health Score Algorithm")
    print(f"Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print("=" * 60)

    cursor = self.db.cursor()
    cursor.execute("SELECT id, hostname, ip_address, site FROM devices")
    devices = cursor.fetchall()

    site_scores = {'HQ': [], 'BranchA': [], 'BranchB': []}

    for device in devices:
        device_id, hostname, ip, site = device
        score = self.calculate_health_score(ip)
        status = 'active' if score > 0 else 'offline'

        cursor.execute("""
            UPDATE devices SET status=%s WHERE id=%s
        """, (status, device_id))

        self.save_to_db(device_id, score, status)

        icon = '✅' if score >= 70 else '⚠️' if score > 0 else '❌'
        print(f"{icon} {hostname:10} | {ip:15} | Score: {score:6}/100 | {status}")

        if site in site_scores:
            site_scores[site].append(score)

    self.db.commit()

    print("\n" + "=" * 60)
    print("SITE HEALTH SUMMARY")
    print("=" * 60)
    for site, scores in site_scores.items():
        if scores:
            avg = round(sum(scores) / len(scores), 2)
            print(f"{site:10}: {avg}/100")

    print("\n✅ Health scores saved to MySQL!")

if __name__ == '__main__':
    health = NetworkHealthAlgorithm()
    health.run()

```

Fig: Weighted Health Scoring Algorithm Implementation